

Machine Learning-Based Prediction of Mini Flash Crashes

Bo Liu*

Department of Economics

Huron University College

Ke Xu[†]

Department of Economics

University of Victoria

Xinwei Zheng[‡]

Department of Finance

Deakin University

May 28, 2025

*I would also like to express my appreciation to Yue Zhao (University of Southern California) for his suggestions on the initial data processing ideas.

[†]Corresponding author. E-mail: kexu@uvic.ca. Ke Xu is grateful to the Social Sciences and Humanities Research Council (SSHRC) Insight Development Grants (430-2018-00557) for financial support.

[‡]E-mail: xinwei.zheng@deakin.edu.au.

Machine Learning-Based Prediction of Mini Flash Crashes

May 28, 2025

1 Introduction

The correlation between high-frequency trading (HFT) and flash crashes has been a prominent topic of discussion in academic literature. A notable instance of such a crash took place on May 6, 2010, when the Dow Jones Industrial Average recorded its largest intraday decline in history, which is named as “Flash Crash”. The screenshot of Bloomberg is shown in Figure 1. While the Securities and Exchange Commission (SEC) and the Commodity Futures Trading Commission (CFTC) have acknowledged the challenge of pinpointing the precise cause of this flash crash, there is a prevalent belief in media and public discourse that one of the primary suspects is the cancellation of existing buy orders by high-speed trading algorithms when they detect market imbalances (Nolte (2010)).

Figure 1

Fosset et al. (2023) highlights the vulnerability of market liquidity to potential death spirals when both human and machine-driven market makers react excessively to unforeseen events. High-frequency traders (HFTs), acting as ultra-fast intermediaries, face susceptibility to significant overexposure in either long or short positions, necessitating swift adjustments. Scholtus et al. (2014) found that a trader with a 100% success rate incurs a loss of 0.81 (1.62) basis points per event, or 1.94% (3.90%) per year, due to a delay of 300 milliseconds (1 second). While HFTs themselves may not be the primary instigators, their algorithms can exacerbate market conditions, potentially leading to flash crashes Bellia (2020).

The study cited in Baron et al. (2019) highlights the pronounced sensitivity of high-frequency traders (HFTs) to speed, driven by their ability to rapidly assimilate short-lived information. This underscores the short-term focus of HFT strategies, particularly in market making and cross-market arbitrage. Complementing this, Li et al. (2020) finds that program trading induces excess return comovement among intensively traded stocks that is unrelated to fundamentals or market news, becomes more pronounced under heightened uncertainty, and supports the theory of habitat investing by revealing a persistent, non-fundamental

source of price co-movement.

In parallel, research on flash crashes, such as Easley et al. (2012), Andersen and Bon-darenko (2014b), and Brogaard et al. (2018), yields divergent conclusions. While some studies refute HFT as the primary cause, others, like Leal et al. (2014), suggest HFTs can exacerbate flash crashes through unique trading strategies.

Scholarly discourse, as in Foucault (2016), underscores concerns about HFT's market impact, citing heightened adverse selection costs and reduced market informativeness. The paper suggests recent market dislocations are more likely due to trading automation and market structural shifts than solely attributable to HFT.

According to Keller (2012)), there is a call for regulators to strike a careful balance, aiming to restore market confidence without undermining the efficiencies brought about by HFT, achieved through heightened transparency and more robust reporting requirements. Moreover, Keller (2012) also advocates for the implementation of internal risk management measures within HFT firms.

Due to the sporadic and unpredictable occurrences of events akin to the flash crash, as exemplified by the incident on May 6th, 2010, researchers have redirected their attention to analyzing smaller-scale flash crashes to glean valuable insights (Nanex (2010)). Various studies have emerged with the objective of estimating mini flash crashes, employing the widely accepted rule-of-thumb definition initially introduced by Nanex (2010).¹

Notably, Golub et al. (2012) have uncovered evidence underscoring the detrimental effect of these mini flash crashes on market liquidity. In line with their discoveries, our research highlights that information related to market liquidity, encompassing trade volume and limit order book data, serves as a particularly robust signal for predicting the occurrence of mini

¹Nanex (2010)'s definition of mini flash crashes stipulates a minimum requirement of 10 consecutive upward or downward ticks occurring within a 1.5-second interval. This criterion differentiates mini flash crashes from classical events termed as "Price Jumps", which are often initiated by information shocks. For an event to be classified as a mini flash crash, it necessitates significant price fluctuations that surpass 10 occurrences. Failing to meet this threshold would render the event outside the scope of the defined criteria.

flash crashes.

Indeed, Golub et al. (2012) have highlighted the detrimental impact of mini flash crashes on market liquidity. Our research further emphasizes the importance of market liquidity indicators, such as trade volume and limit order book data, in predicting these events.

Moreover, Kirilenko et al. (2017) investigated the trading behaviors of High-Frequency Traders (HFTs) during the Flash Crash, revealing distinctive patterns. In our study, we utilize machine learning techniques to identify clear patterns in market conditions, providing evidence of specific participant behaviors during mini flash crash events.

Figure 2

In this paper, we adopt the framework established by Aït-Sahalia et al. (2022) and draw upon the insights of Easley et al. (2021) to evaluate the predictability of efficient information and microstructure measures across short- and long-time horizons. Our empirical findings demonstrate that machine learning effectively anticipates mini flash crashes by leveraging real-time market data. Notably, short-term information related to trade volume and transaction counts exhibits robust predictive power, reflecting market fluctuations through trading behaviors.

In addition to the domains identified by Easley (2013), we highlight another promising application area for machine learning: the detection of anomalous events, including mini flash crashes. Building upon their insights, our research underscores machine learning's proficiency in identifying and predicting these events. As emphasized by Easley et al. (2021), market microstructure information remains a valuable source of predictability even in the era of machine learning. Our prediction model heavily relies on the microstructure data highlighted by Easley et al. (2021).

This study adds valuable insights to the literature by exploring novel perspectives on market data, building on the groundwork laid by O'Hara (2015). Furthermore, we consider this paper as an initial step towards the application of machine learning for the prediction of mini flash crashes.

Relying on pattern recognition approaches, this paper makes a significant contribution by demonstrating that, unlike long-term stock returns, short-term events such as mini flash crashes are universally predictable without requiring specific asset-level information. In a related context, Rad et al. (2023) finds that machine learning models, particularly those incorporating nonlinear and best capture commodity futures risk premia among 128 predictors, underscoring the value of flexible model architectures for robust out-of-sample performance beyond traditional risk factors.

The contributions of this paper can be viewed through three key angles. To begin, we provide compelling evidence that machine learning can be effectively leveraged for order flow anomaly detection, extending its application beyond areas like optimized trade execution, price movement forecasting, and execution optimization in dark pools. This highlights the versatility of machine learning in financial analysis.

Additionally, we show that even in the case of mini flash crashes, which are events that defy interpretation by linear models, ensemble machine learning models can reliably predict them. This highlights the strength of machine learning in capturing complex, non-linear relationships within financial data.

Finally, we empirically demonstrate that certain critical features, particularly those tied to market liquidity and informed trading measures, exhibit strong real-time predictive capabilities in relation to flash crashes. These findings suggest the potential involvement of market makers and informed traders in flash crash events, offering new insights into the dynamics driving these occurrences.

To explore our findings comprehensively, we've structured the paper into six sections. After the introduction, the paper unfolds as follows: Section 2 reviews institutional background. Section 3 details our methodology, covering problem formulation, machine learning models, accuracy assessment, and data imbalance mitigation. Section 4 outlines our dataset and preprocessing approach. In Section 5, we present our empirical results and their implications. Finally, Section 7 offers concluding remarks on our findings.

2 Institutional Background

Following the May 6, 2010, flash crash, regulatory authorities worldwide implemented significant measures to mitigate the risk of similar events in the future. In the U.S., the Securities and Exchange Commission (SEC) introduced Stock-by-Stock Circuit Breaker rules that trigger a temporary trading halt for stocks experiencing a 10 percent price change within five minutes. While these circuit breakers provide some protection against market instability, they are not without drawbacks. For instance, research by Wang et al. (2022) highlighted the adverse effects of market-wide circuit breakers implemented by Chinese stock market regulators on January 4, 2016. The uncertain and disruptive impact of these breakers led to widespread panic selling and intensified herding behavior, underscoring the delicate balance regulators must maintain in preventing systemic disruptions while avoiding market overreactions.²

Organizations such as the Financial Stability Board (FSB) and the International Organization of Securities Commissions (IOSCO) have acknowledged the systemic risks associated with mini flash crashes. IOSCO's 2011 report on the U.S. flash crash revealed that over 20,000 trades across 300 securities occurred at prices deviating by as much as 60% from their recent values, highlighting the global nature of such risks. The interconnectedness of financial markets means that a mini flash crash in one region can have ripple effects across borders, prompting calls for greater international regulatory cooperation and data sharing.(OICU-IOSCO (2011))

Institutional investors, particularly mutual funds, are especially vulnerable to mini flash crashes. Many funds utilize fire-sale (stop-loss) algorithms to protect their portfolios, which can inadvertently trigger large-scale sell-offs when a mini flash crash occurs. Dyakov and Ver-

²Although it is widely recognized that China's emerging capital markets are significantly different from the more mature markets of the United States, the severe disruptions and extensive debates triggered by the implementation of circuit breakers in the Chinese stock market provide grounds to believe that relying solely on circuit breakers to address the systemic risks associated with flash crashes is both unreliable and uncontrollable.

beek (2013) demonstrated how mini flash crashes can be exploited through trading strategies that anticipate mutual fund fire-sales, creating additional risks for public investors. Furthermore, Deng et al. (2018) found that mutual fund herding behavior amplifies the risk of stock price crashes, exacerbating the financial vulnerabilities posed by these events.

Ultimately, these regulatory efforts reflect a growing recognition of the complexities introduced by modern trading technologies. As markets evolve, so too must the tools and frameworks employed to monitor them. By understanding the triggers and mechanisms behind mini flash crashes, regulators and market participants can better navigate the challenges posed by algorithmic trading, reduce reliance on reactive measures like circuit breakers, and strengthen financial stability on a global scale.

3 Methodology, Specification and Data

In this section, we provide a detailed explanation of our methodologies, which encompass the predictor variables employed, the machine learning models utilized, the approach used to measure prediction accuracy, and our strategies for handling imbalanced data.

3.1 Predictor Variables

In forecasting the foregoing response variables, a diverse array of predictor features is integrated into the model's framework. The construction of most of these predictor variables closely adheres to the methodology originally proposed by Aït-Sahalia et al. (2022),), encompassing numerous derived variables characterized by nonlinear transformations of historical data, particularly when using fine-grained time intervals. Building on the insights gleaned from established works in the field, such as those by Cont et al. (2013) and Kercheval and Zhang (2015), we anticipate that the pivotal determinants for predicting forthcoming short-term events, such as mini flash crashes, will pivot around the characteristics of the prevailing Limit Order Book (LOB).

Mase (1999) also points out liquidity effects can provide prediction ability on short horizon return. These attributes encompass potential imbalances and historical trade returns at the point of prediction. Menkhoff et al. (2010) highlights that informed traders are highly sensitive to factors such as spreads, volatility, momentum, and market depth. Additionally, in consonance with the approach advanced by Aït-Sahalia et al. (2022), we incorporate microstructure measures as expounded upon by Easley et al. (2021). This integration captures information related to market microstructure noise, which may harbor significant signals of unforeseen events. Notably, three supplementary predictors are introduced, with a specific focus on microstructure noise. Comprehensive descriptions of all these features will be explained in subsequent sections.

In parallel with the forward-looking intervals that are considered in relation to the response variable, namely, mini flash crashes, an analogous approach is adopted to establish look-back intervals based on calendar time. This entails employing the current timestamp, denoted as T , and defining lookback spans as (Δ_1, Δ_2) . For the derivation of predictor variables, a set of lookback windows is designated as $I = Int(T - \Delta_2, T - \Delta_1)$. More specifically, the values assigned to (Δ_1, Δ_2) span a spectrum from $(0s, 0.1s)$ to $(102.4s, 204.8s)$, contains time interval from shortest to longest, no overlap to isolate information. The most extensive span, which is $204.8s$, effectively encompasses a slightly extended 3-minute horizon beyond each timestamp T . This strategic choice ensures that the prediction model avoids reliance on transient information for forecasting longer-term outcomes. In total, our framework encompasses 11 distinct look-back windows, where features are computed directly upon the conclusion of each respective interval.

Let D^{txn} represent the set of all timestamps t of trade transactions within the given time interval, and D^{qt} representing its counterpart for quote data. The combined set is denoted as $D = D^{txn} \cup D^{qt}$. Within this dataset, the National Best Bid and Offer (NBBO) prices are indexed by $t \in D$ and expressed as (P_t^b, P_t^a) , where P_t^b represents the best bid price, and P_t^a designates the best ask price. The mid-price is computed as the simple average of these

two, denoted as $P_t = \frac{P_t^b + P_t^a}{2}$.

Further, if t belongs to the set D^{txn} , we denote the transacted price as P_t^{txn} . The best bid and ask sizes are also represented as S_t^b and S_t^a respectively, pertaining to the record indexed by t .

Volume and duration: Predictors in this category are linked to a stock's recent trading activity. For instance, block trades or frequent transactions may indicate increased short-term trading activity. While this heightened activity doesn't inherently reveal the trend's direction, it can interact nonlinearly with other predictors, potentially strengthening a trend's emergence.

Breadth measures the number of transactions in the interval:

$$Breath(T, \Delta_1, \Delta_2) = |D^{txn} \cap Int^{back}(T, \Delta_1, \Delta_2)| \quad (1)$$

Immediacy measures the average time between successive transactions in the interval:

$$Immediacy(T, \Delta_1, \Delta_2) = \frac{\Delta_1 - \Delta_2}{Breath(T, \Delta_1, \Delta_2)} \quad (2)$$

VolumeAll measures the total number of shares transacted in the interval:

$$VolumeAll(T, \Delta_1, \Delta_2) = \sum_{t \in Int^{back}(T, \Delta_1, \Delta_2)} V_t \quad (3)$$

VolumeAvg measures the average number of shares transacted for each transaction in the interval:

$$VolumeAvg(T, \Delta_1, \Delta_2) = \frac{VolumeAll(T, \Delta_1)}{Breath(T, \Delta_1, \Delta_2)} \quad (4)$$

VolumeMax measures the maximum number of shares transacted in one transaction in the interval:

$$VolumeMax(T, \Delta_1, \Delta_2) = \max\{V_t : t \in Int^{back}(T, \Delta_1, \Delta_2)\} \quad (5)$$

Return and imbalances: These predictors relate to recent trading imbalances within the stock, providing insights into short-term trends. These trends are discerned through the analysis of trade transactions and quotes data. For instance, a significant volume of buying leading to triggering of limit sell orders, or a consistently higher bid compared to the ask in Level I quotes, indicates a potential upward price influence. To capture these dynamics, the following variables are outlined.

Lambda measures the price change in the interval proportional to total volume.

Let $I = D^{txn} \cap Int^{back}(T, \Delta_1, \Delta_2)$, then $P_{max(I)}$ and $P_{min(I)}$ represent the Maximum and Minimum transaction price I , then:

$$Lambda(T, \Delta_1, \Delta_2) = \frac{P_{max(I)} - P_{min(I)}}{VolumeAll(T, \Delta_1, \Delta_2)} \quad (6)$$

LobImbalance is the average imbalance in the depth of the limit order book over the lookback interval:

$$LobImbalance(T, \Delta_1, \Delta_2) = Average\left[\frac{S_t^a - S_t^b}{S_t^a + S_t^b} : t \in Int^{back}(T, \Delta_1, \Delta_2)\right] \quad (7)$$

TxnImbalance measures the asymmetry of buy and sells volumes in recent transactions. Denote by Dir_t^{LR} the binary transaction direction at time t signed using the algorithm of Lee and Ready (1991). Then transaction imbalance is calculated as

$$TxnImbalance(T, \Delta_1, \Delta_2) = Average\left[\frac{\sum_{t \in D^{txn} \cap Int^{back}(T, \Delta_1, \Delta_2)} (V_t Dir_t^{LR})}{VolumeAll(T, \Delta_1, \Delta_2)}\right] \quad (8)$$

PastReturn is the past return in the lookback window. Let $I = D^{txn} \cap Int^{back}(T, \Delta_1, \Delta_2)$:

$$PastReturn(T, \Delta_1, \Delta_2) = 1 - Average\left[\frac{P_t^{txn} : t \in I}{P_{max(I)}}\right] \quad (9)$$

Speed and cost: This set of predictors we employ measure the speed and cost inherent in the stock's trading.

Turnover is the speed of transactions to the stock's total number of shares outstanding.

$$Turnover(T, \Delta_1, \Delta_2) = \frac{VolumeAll(T, \Delta_1, \Delta_2)}{S} \quad (10)$$

AutoCov is the autocovariance of transaction returns in the interval. For any $t \in D^{txn}$, denote by $L_t = \operatorname{argmax}_s \{s : s < t, s \in D^{txn}\}$ the timestamp of the transaction right before time t . Then the autocovariance is:

$$AutoCov(T, \Delta_1, \Delta_2) = \text{Average} \left[\log \left(\frac{P_t^{txn}}{P_{L_t}^{txn}} \right) \log \left(\frac{P_t^{txn}}{P_{L(L_t)}^{txn}} \right) : t \in D^{txn} \cap \text{Int } t^{\text{back}}(T, \Delta_1, \Delta_2) \right] \quad (11)$$

QuotedSpread is the average proportional nominal spread in the quotes over the lookback interval:

$$QuotedSpread(T, \Delta_1, \Delta_2) = \text{Average} \left[\frac{P_t^a - P_t^b}{P_t} : t \in \text{Int}^{\text{back}}(T, \Delta_1, \Delta_2) \right] \quad (12)$$

EffectiveSpread is the dollar-weighted percent effective spread over the interval:

$$EffectiveSpread(T, \Delta_1, \Delta_2) = \frac{\sum_{t \in D^{txn} \cap \text{Int}^{\text{back}}(T, \Delta_1, \Delta_2)} \left[\log \left(\frac{P_t^{txn}}{P_t} \right) \text{Dir}_t^{LR} V_t P_t^{txn} \right]}{\sum_{t \in D^{txn} \cap \text{Int}^{\text{back}}(T, \Delta_1, \Delta_2)} (V_t P_t^{txn})} \quad (13)$$

Microstructure Measures As outlined by Easley et al. (2021), this set of measures comprises widely recognized market microstructure variables that have gained prominence, particularly following the May 6, 2010 “flash crash.” With the growing need for early warning signals of impending market stress, the use of such measures to predict real-time turbulence has become increasingly attractive, though some remain subject to scrutiny Andersen and

Bondarenko (2014a). Chiu and Chen (2023) shows that investor sophistication influences limit order behavior in the Taiwan futures market: institutional investors actively cancel and revise orders to manage free-option risk, while individual investors are less responsive. Informational volatility further affects order modifications across all investor types. We will provide additional evidence to assess the suitability of these measures. As mentioned earlier, the effectiveness of forecasting mini flash crashes depends on the model's real-time predictive capability. Therefore, we compute all microstructure variables using backward-looking windows. Specifically, we include:

Roll measure:

$$\begin{aligned} R_t &= 2\sqrt{|\text{cov}(\Delta \mathbf{P}_t, \Delta \mathbf{P}_{t-1})|} \\ \Delta \mathbf{P}_t &= [\Delta P_{t-w}, \Delta P_{t-w-1}, \dots, \Delta P_t], \\ \Delta \mathbf{P}_{t-1} &= [\Delta P_{t-w-1}, \Delta P_{t-w}, \dots, \Delta P_{t-1}], \end{aligned} \quad (14)$$

Where ΔP_t is the change in close price between bars $t-1$ and t and W is the lookback window size.

Roll impact, which is the Roll measure divided by the value traded over the lookback window, is:

$$\tilde{R}_t = \frac{2\sqrt{|\text{cov}(\Delta \mathbf{P}_t, \Delta \mathbf{P}_{t-1})|}}{P_t V_t} \quad (15)$$

Kyle's lambda is given by:

$$\lambda_t = \frac{P_t - P_{t-w}}{\sum_{i=t}^t b_i V_t} \quad (16)$$

Where b_i is the trade indicator inferred by Lee and Ready (1991), which is computed through one lookback window.

Amihud's measure:

$$\lambda_t^A = \frac{1}{W} \sum_{i=t-W+1}^t \frac{|r_i|}{p_i V_i} \quad (17)$$

Where r_i, p_i, V_i are the return, price, and volume at look back window i and W is the lookback window size in terms of the number of trades.

Volume-synchronized probability of informed trading is estimated as:

$$\text{VPIN}_t = \frac{1}{W} \sum_{i=\tau-W+1}^{\tau} \frac{|P_t^a - P_t^b|}{V_i} \quad (18)$$

P_t^a and P_t^b are bid and ask quotes.

Microstructure Realized Volatility and Noise The final set of measures builds on the literature surrounding microstructure noise, consistent with the theoretical insights of Zhang et al. (2005). Studies such as Bossaerts et al. (2013) show that price volatility increases as information is incorporated into prices. Watanabe and Nakajima (2024) develops a high-frequency realized stochastic volatility model that accounts for intraday volatility patterns, microstructure noise bias, and macroeconomic announcements, and demonstrates—via Bayesian estimation—that the model improves in-sample fit and volatility forecasting using 5-minute E-mini S&P 500 futures data. In a related direction, Zhu et al. (2023) proposes a panel-data-based machine learning (PDML) approach for forecasting realized volatility with high-frequency data, showing that PDML outperforms both traditional linear models and single-series machine learning forecasts in short-term prediction accuracy. Since short-term volatility can be highly informative for real-time price prediction, we introduce the following three measures:

Realized Volatility in each lookback window:

$$[P, P]_w = \sum_{t \in W} (P_{t_{i+1}} - P_{t_i})^2 \quad (19)$$

Two-Scales Realized Volatility (TSRV) in each lookback window:

$$\widehat{\langle P, P \rangle}_T = [Y, Y]_T^{\text{avg}} - \frac{\bar{n}}{n} [Y, Y]_T^{\text{all}} \quad (20)$$

The combination is of two time scales, “all” and “average” sampling. More details could be found in Zhang et al. (2005). The third one is *Realizedmoments of Disjoint Increments (ReMeDI)* which is a new-developed measure to estimate microstructure noise, see Li and Linton (2022).

VIX: The last one is the daily Volatility Index (VIX) of the previous trading day.

3.2 Machine Learning Methods

3.2.1 Models

This paper employs four primary machine learning models for the prediction of mini flash crashes. These models encompass a regularized logistic regression (LASSO) as a representative of a linear parametric approach, alongside a penalized support vector machine (penalized-SVM) offering a nonparametric alternative. Additionally, two ensemble models, namely Random Forest and Extreme Gradient Boosting (XGBoost), are utilized. Detailed theoretical expositions of these models can be found in the works of Hastie et al. (2009) and Murphy (2013).

Consider the challenge of imbalanced classification, where the goal is to predict a response variable Y . Here, Y takes the value 1 when a mini flash crash is anticipated within the designated look-forward window, and 0 when no mini flash crash is expected. This prediction is based on a predictor vector X , utilizing a random sample (X_i, Y_i) . We can represent the response vector as $\mathbf{Y} = (y_1, \dots, y_n)^T$. In our feature vectors, X_i , 232 dimensions, encompassing 11 time spans for each of the 21 predictor variables, with the final dimension representing the volatility index (VIX) from the previous trading day. Machine learning algorithms inherently have the capability to generate additional combinations of these predictors or select the most informative subsets from them.

3.2.2 Penalized Logistic Regression (LR)

Within the realm of well-established machine learning models, linear models stand out for their unique appeal stemming from their simplicity and high interpretability. Logistic regression, a classification model, leverages the sigmoid function to map the value range of linear regression into the interval $[0, 1]$. In this context, the response variable is formulated as follows:

$$y = \frac{1}{1 + e^{-z}} \quad (21)$$

Through linear regression model:

$$Z = \beta^T \mathbf{X} + \epsilon \quad (22)$$

$\mathbf{X}$ represents the predictor variables. In the absence of some form of regularization, standard OLS in a large dimensional setting is likely to have poor out-of-sample predictive power due to in-sample overfitting. A standard method to address this issue consists in regularizing the model using a penalty function applied to normalized variables. Penalized least squares with an L_1 penalty is recognized as the Least Absolute Shrinkage and Selection Operator (LASSO). Specifically, consider $\bar{X} = \frac{1}{n} \sum_i X_i$ and $s_i = \sqrt{\frac{1}{n} \sum_i (x_i - \bar{x}_i)^2}$, representing the mean vector and standard deviations of predictor variables, respectively. Let $\bar{Z} = \frac{1}{n} \sum_i Z_i$ denote the mean of the linear regression response variable. Define the centered regression response as $\tilde{Z}_i = Z_i - \bar{Z}$ and standardized predictors as $\tilde{\mathbf{X}}_i = \text{diag}(s_1^{-1}, s_2^{-1} \dots s_p^{-1}) (\mathbf{X}_i - \bar{\mathbf{X}})$ (for $i = 1, \dots, n$). LASSO proceeds to fit the centered response onto the standardized predictors by solving the ensuing optimization problem:

$$\hat{\beta} = \underset{\beta \in R^p}{\text{argmin}} \left\{ \frac{1}{n} \sum_i \left(\tilde{Z}_i - \beta^T \tilde{\mathbf{X}}_i \right)^2 + \lambda \|\beta\| \right\} \quad (23)$$

This optimization problem could be solved by convex optimization. In this paper, we use

the coordinate descent algorithm implemented in the Scikit-learn software by Python.

After I solve the coefficient $\hat{\beta}$, I can predict each new data $\mathbf{X}_{new}$ as:

$$\widehat{Y}_{new} = \frac{1}{1 + e^{-(\bar{\mathbf{z}} + \hat{\beta}^T \mathbf{X}_{new})}}, \text{ with } \widetilde{\mathbf{X}_{new}} = \text{diag}(s_1^{-1}, s_2^{-1}, \dots, s_p^{-1}) (\mathbf{X}_{new} - \bar{\mathbf{X}}) \quad (24)$$

LASSO is a simple and very easily explained model that shrinks the coefficients of less useful predictors towards zero. This can help us to rank the relevance of different predictors in the prediction problem. Through this process, we can summarize the feature that can show significant power to become the signal of mini flash crashes.

3.2.3 Support Vector Machine (SVM)

Support vector machine (SVM) is one of the most popular non-parametric algorithms for its outstanding performance and clear mathematical details. We use it as the representative of the non-parametric forecasting model. In this section, we briefly introduce SVM; more details may be found in Hastie et al. (2009) and Murphy (2013).

A support vector machine constructs a hyper-plane or set of hyper-planes in a high or infinite-dimensional space, which can be used for both classification and regression tasks.

Giving training vectors $x_i \in R^p, i = 1, \dots, n$ in two classes, and the response variable as $y \in \{1, -1\}^n$, we want to find a $\omega \in R^p$ and $b \in R$ such that $\omega^T \phi(x) + b$ can predict the sign of most x_{new} .

Having this ω , SVM solves the following primal optimization problem:

$$\min_{\omega, b, \zeta} \frac{1}{2} \omega^T \omega + C \sum_{i=1}^n \zeta_i \quad (25)$$

$$\text{Subject to } y_i (\omega^T \phi(x) + b) \geq 1 - \zeta_i, \zeta_i \geq 0, i = 1, \dots, n$$

Through the SVM, we are trying to maximize the margin between two classes by minimizing $|\omega|^2 = \omega^T \omega$. The perfect situation is the hyperplane that can separate all samples,

which are $y_i (\omega^T \phi(x_i) + b) \geq 1$ for all samples. However, in the real world, samples are usually not perfectly separable by a hyperplane; we must allow some samples for the generalizing ability of the model. Therefore, some samples need to be at a distance denoted as ζ_i , from their correct margin boundary. The penalty term C controls the strength of this penalty. The dual problem to the primal problem is:

$$\min_{\alpha} \frac{1}{2} \alpha^T Q \alpha - e^T \alpha$$

Subject to $y^T \alpha = 0$ and $0 \leq \alpha_i \leq C$ for $i = 1, \dots, n$, where e is the vector of all ones, and Q is the matrix: $Q_{ij} \equiv y_i y_j K(x_i, x_j)$, with $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$ is the kernel.

The kernel function can map the samples into higher dimensional or infinite-dimensional space; for more details see Hastie et al. (2009).

In this paper's mini flash crash forecasting problem, we have conducted many iterations to tune the hyper-parameters, determining that using a linear kernel (simple inner product $\langle x, x \rangle$) and hinge loss to construct the optimization problem, can significantly outperform all other kernels. In the empirical results, we will simply show the linear kernel support vector machine results.

When the optimization problem is solved, we can use a support vector to predict a new sample x_{new} by:

$$\sum_{i \in SV}^n y_i \alpha_i K(x_i, x_{new}) + b \quad (26)$$

Then the predicted class corresponds to its sign.

3.2.4 Random forests (RF)

While lacking the interpretability of linear models, ensemble learning tree-style models exhibit impressive forecasting abilities in various practical scenarios. In this paper, we employ two ensemble models, starting with a random forest. Random forest, a scalable nonpara-

metric learning method, is built upon individual decision trees. Given the instability and limited predictive power of a single decision tree, constructing a forest of many trees serves as a direct approach to enhance predictive accuracy. By averaging outcomes from numerous sampled decision trees, the variance of predictions decreases, resulting in more stable and reliable forecasts.

According to Hastie et al. (2009)), random forests are fitted via iteratively growing regression trees through the bootstrap sampling process from the given data set. Hastie et al. (2009) describe the algorithm as follows:

- For $b = 1$ to B
 - Draw a bootstrap sample Z^* of size N from the training data.
 - Grow a random-forest tree T_b to the bootstrapped data, by recursively repeating the following steps for each terminal node of the tree, until the minimum node size $n_{\min}$ is reached.
 - * Select m variables at random from the p variables.
 - * Pick the best variable/split-point among the m .
 - * Split the node into two daughter nodes.
 - Output the ensemble of trees $\{T_b\}_1^B$.

For new data, we want to predict x : Let $\hat{C}_b(x)$ be the class prediction of the b th random-forest tree. Then $\hat{C}_{rf}^B(x) = \text{majority vote } \left\{ \hat{C}_b(x) \right\}_1^B$. (Hastie et al. (2009)).

Predictions from bagging are often highly correlated as the samples are from the same data set. Through the bagging of many independently trained decision trees from bootstrapped samples, but no selection of variables at the decision tree's node, the random forest can achieve better performance as it increases the independence of resulting trees and reduces the dependence of the prediction. Through the random forest, we could obtain a smaller variance than a single decision tree, thus obtaining more reliable predictions.

3.2.5 Extreme Gradient Boosting (XGB)

The second ensemble model we use is the extreme gradient boosting (XGBoost), which is from Friedman (2001). Out of the world of deep learning, XGBoost has been the most popular algorithm to impress the machine learning community in the past few years. As an ensemble model, XGBoost relies on many weak-based tree learners. Those weak learners' biases are high, and their predictive power is just slightly better than random guessing.

In contrast to bagging techniques such as Random Forest, in which trees are grown to their maximum extent, boosting attempts to create small trees, which are not deep and easy to interpret. XGBoost starts from initial model F_0 to predict the target variable y , and the residual $y - F_0$. The new model h_1 is fit to the residual, through the combination of h_1 and F_0 , and the mean squared error will be reduced. Then update $F_1(x) \leftarrow F_0(x) + h_1(x)$, until the residuals are minimized as much as possible.

The algorithm proceeds as follows: Given training set $\{(x_i, y_i)\}_{i=1}^N$, and a well-defined differentiable loss function $L(y, F(x))$, several weak tree learners M and a learning rate α .

- Initialize model with a constant:

$$f_0(x) = \widehat{\argmin} \sum_{l=1}^N L(y_l, \theta). \quad (27)$$

- For $m = 1$ to M :

- Compute the gradients and Hessians:

$$\begin{aligned} \hat{g}_m(x_i) &= \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f(x)=f_{(m-1)}(x)} \\ \hat{h}_m(x_i) &= \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)^2} \right]_{f(x)=f_{(m-1)}(x)} \end{aligned} \quad (28)$$

- Fit a weak tree learner using the training set $\left\{x_i, -\frac{\hat{g}_m(x_i)}{\hat{h}_m(x_i)}\right\}_{i=1}^N$ by solve the opti-

mization problem:

$$\begin{aligned}\hat{\phi}_m &= \operatorname{argmin}_{\phi} \sum_{i=1}^N \hat{h}_m(x_i) \left[-\frac{\hat{g}_m(x_i)}{\hat{h}_m(x_i)} - \phi(x_i) \right]^2 \\ \hat{f}_{(m)}(x) &= \propto \hat{\phi}_m(x)\end{aligned}\tag{29}$$

– Update the model:

$$\hat{f}_{(m)}(x) = \hat{f}_{(m-1)}(x) + \hat{f}_{(m)}(x)$$

- Output $\hat{f}(x) = \sum_{m=0}^M \hat{f}_{(m)}(x)$

More details could be found in Friedman (2001).

3.2.6 Measuring Prediction Accuracy

We use Receiver Operating Characteristic (ROC) and Area under the ROC Curve (AUC) metrics to evaluate our model's predictive accuracy. Predicting mini flash crashes resembles a supervised anomaly detection problem, where such occurrences are rare, leading to significant dataset imbalance. In a hypothetical scenario where only 0.1% of look-forward windows are labeled as mini flash crashes, predicting no crashes would yield a misleadingly high accuracy score of 99.9%.

In such cases, ROC provides a more suitable measure for evaluating prediction accuracy in imbalanced datasets. Originating from signal analysis technology developed during World War II, ROC has wide application, including medical issue detection. It comprehensively depicts a classification model's performance across all thresholds, graphing two key parameters.

- True Positive Rate (TPR): $TPR = \frac{TP}{TP+FN}$
- False Positive Rate (FPR): $FPR = \frac{FP}{FP+TN}$

TP, TN, FN, and TN are all from the confusion matrix, see Figure 3:

Figure 3

An ROC curve visually depicts the relationship between True Positive Rate (TPR) and False Positive Rate (FPR) across various classification thresholds. Lowering the threshold increases both False Positives and True Positives. The curve organizes test samples by predicted probability of being True, enabling sequential prediction and calculation of TPR and FPR. Well-performing models exhibit an initially steep ascent followed by a gradual rise.

To address the issue where the model may not differentiate between high and low probabilities of positive predictions, we use a probability-based ROC approach. This involves ranking the predicted probabilities from highest to lowest and then plotting the ROC curve.

AUC quantifies the entire area beneath the ROC curve, ranging from (0,0) to (1,1), akin to calculating the integral of a function. It offers a comprehensive performance measure across all thresholds, representing the probability of ranking a randomly chosen positive example higher than a randomly chosen negative one. Figure 4 shows the ROC curve of our best-tuned model for 15s mini flash crashes in Window 7, covering July to September 2018.

Figure 4

3.2.7 Imbalanced Data Processing Strategies

As previously mentioned, our problem entails dealing with imbalanced data. In addition to utilizing the raw data as-is, we will also implement five distinct strategies to mitigate the data imbalance issue.

Undersampling (UD): One method to tackle data imbalance involves removing excess samples from the majority class, specifically those where the look-forward window doesn't result in a mini flash crash, to establish a balanced sample set relative to the minority class. While straightforward, this approach results in significant data loss from the majority class, potentially weakening its impact, especially for instances offering crucial information between classes. For example, in support vector machine algorithms, removed samples may include

support vectors near the margin hyperplane on the majority class side, potentially further biasing the model.

Oversampling (OV): Another strategy involves randomly duplicating synthetic samples within the minority class, particularly when the look-forward window results in a mini flash crash. This approach aims to balance the class distribution by creating additional synthetic samples. However, it's important to note that this strategy carries potential risks. Introducing noise samples into the minority class may amplify noise presence and lead to overfitting the model.

Synthetic Minority Oversampling Technique (SMOTE): SMOTE is a sophisticated oversampling technique that utilizes the K-Nearest Neighbors (KNN) algorithm to generate new samples for the minority class. These new samples mimic the originals while introducing some variation. Compared to basic oversampling, SMOTE functions as an ensemble learning method, potentially reducing variance and overfitting risk. However, it relies on the minority class to create synthetic samples, which might amplify noise due to shifts in data distribution. Moreover, SMOTE is computationally more complex than both oversampling and undersampling techniques.

Threshold Moving (TM): This strategy involves adjusting the classification threshold to increase the model's sensitivity to the minority class. In balanced data scenarios, the threshold typically remains at 0.5. However, in imbalanced data situations, the threshold can be modified to align with the ratio of minority to majority class samples within the training dataset.

Ensemble Undersampling (EN): As previously discussed, undersampling removes numerous majority class samples to balance the dataset, leading to substantial data loss. An alternative involves multiple rounds of undersampling. For instance, with 10,000 majority class samples and 50 minority class samples, 200 new sets can be created by selecting all 50 minority samples and randomly choosing an additional 50 for each set. These 200 models are trained independently, and their results are combined for the final prediction.

This strategy has limitations, including increased computational complexity and potential overfitting due to repeated use of minority class samples. However, in anomaly detection scenarios like ours, where successfully detecting mini flash crashes is paramount, this strategy proves invaluable with high-quality minority class data.

4 Data description and pre-processing

In this paper, we use all 95 stocks in S&P 100 from the NYSE Tick and Quote (TAQ) data set. The time range of data covers two years, 2017 and 2018. Following Hendershott and Moulton (2011), we use the same standard to clean the quote and transaction data.³

To enhance the predictive robustness of our model, we conducted a meticulous segmentation of the two-year dataset, partitioning it into eight discrete windows, each spanning a three-month period. Within each window, a structured approach was employed: the first month was allocated for training the model, the second month for the fine-tuning of parameters aimed at identifying the most optimal configuration, and, subsequently, the third month was reserved for assessing the model's performance.

This study utilizes short-term, high-frequency transaction and quote data to predict mini flash crashes. Unlike some existing research, which often aggregates data using event windows or information bars, exemplified by Easley et al. (2021), we adopt a temporal framework, relying on a Time clock approach.

In practice, portfolio managers facing an imminent mini flash crash require real-time predictions rather than waiting for additional events to accumulate. Therefore, to ensure real-time predictive capability, we exclusively employ a temporal framework for detecting

³When we do the data filtering, we restricted the trading data in regular trading hours from 9:30 am to 4 pm. We use only trades for which TAQ's CORR field is zero, one, or two and for COND field is either blank or equal to @, E, F, I, J, or K. Obviously, we eliminate trades with nonpositive prices or quantities. We also remove trades with prices more than (less than) 150% (50%) of the previous trade price. After that, we restrict quotes for which TAQ's MODE field is equal to 1, 2, 6, 10, 12, 21, 22, 23, 24, 25, or 26. Then we eliminate quotes with nonpositive prices or sizes or with bid prices greater than the asking price. We also exclude quotes when the quoted is greater than 25% of the quote midpoint or when the asking price is more than 150% of the bid price.

relevant variables.

Mini flash crashes have been extensively investigated in the literature, with the most accepted definition originating from Nanex (2010). According to Nanex (2010), mini flash crashes are succinctly defined as follows:

- The time window does not exceed 1.5 seconds;
- Price change exceeds 0.8%;
- At least 10 times tick down before ticking up OR at least 10 times tick up before ticking down;

In this paper, three additional, more lenient time window definitions are introduced: 180 seconds, 90 seconds, and 15 seconds, in conjunction with the Nanex (2010)'s established 1.5-second definition. These four discrete time intervals function as distinct criteria for the analysis and categorization of mini flash crashes, accounting for a broad spectrum of time frames and accommodating the diverse operational velocities of traders.

For individual investors and slower-paced traders, the 180-second time window provides ample time for trading activities to unfold. Mini flash crashes within this interval encompass a range of trader speeds and behaviors. In contrast, the 90-second timeframe represents a medium-term window where slower traders may still influence market dynamics to some extent. However, mini flash crashes within this timeframe are mainly influenced by fast-paced traders, with possible participation from slower traders.

The third temporal definition used is 15 seconds, a brief timeframe where individual traders are unlikely to significantly impact market dynamics. However, it's crucial to note that machine-executed trading algorithms vary in speed. While some quantitative trading firms use automated systems that respond within seconds, high-frequency traders (HFTs) operate at a much faster pace, executing trades at the nanosecond level. This study aligns

with Nanex (2010)’s 1.5-second definition, focusing exclusively on mini flash crashes involving extremely rapid trading algorithms.

In addition to Nanex (2010)’s definition, Dugast and Foucault (2018) define mini flash crashes as large sudden price drops or spikes followed by quick price reversals, i.e. “V-shape” or “inverted V-shape” price movements. Inspired by this idea, a reverse version of mini flash crashes is implemented:

- The time window does not exceed 180 seconds/90 seconds/15 seconds/1.5 seconds;
- Price change exceeds 0.8%;
- At least 10 times tick down before ticking up AND at least 10 times tick up before ticking down;

In pursuing the capture of the entire spectrum of mini flash crashes and in the endeavor to construct a real-time forecasting model, a comprehensive approach is undertaken. This approach entails considering all possible time intervals within each trading day. To achieve this, a forward-looking window is employed for the detection of mini flash crashes:

$$\text{Int}^{\text{forward}}(T, T + \Delta) = \{t \in R : T < t \leq T + \Delta\}, \text{ a span } \Delta \text{ equals } 180\text{s}/90\text{s}/15\text{s}/1.5\text{s}.$$

Table 1 shows the descriptive statistics of all the windows, covering the time range from January 2017 to December 2018, including four different definitions of mini flash crashes without reversal.

Table 1

Table 2 shows the descriptive statistics of all of our windows, cover time range from January 2017 to December 2018 includes four different definitions of mini flash crashes with reversal. For both Table 1 and Table 2, we removed time intervals that lacked any information, including empty bid, ask, and transaction data within the interval and across all look-back windows.

Table 2

5 Empirical Analysis

In addition to using the original imbalanced data, all previously discussed strategies were applied alongside four models: logistic regression with an l_1 penalty (LASSO), random forest, support vector machine, and XGBoost. As outlined in Section 3, we investigate eight distinct scenarios concerning mini flash crashes based on different look-forward windows (180s/90s/15s/1.5s) and reversal criteria, as defined in Section 4.

Inspired by Nanex (2010), this paper adopts a concise 1.5-second time window to define mini flash crashes. This duration is chosen to account for the swift pace of trading activity, where even slower traders cannot react quickly enough to significantly impact stock prices. To provide further insights, we introduce three additional time windows: 180 seconds (long-term), 90 seconds (medium-term), and 15 seconds (short-term).

Aligned with Nanex (2010), the extremely short-term category is the strictest classification of mini flash crashes. Within the 15-second timeframe, the short-term perspective primarily reflects fast traders' influence, offering insights into overall market conditions. Moving to the medium term, a robust check on the long-term viewpoint is established, revealing the involvement of fast-reactive slow traders in these events. Finally, mini flash crashes unfold over an extended 180-second window in the long term, allowing both slow and fast traders ample time to react to market dynamics.

As outlined in section 3, this paper employs ROC and AUC as the evaluation metrics to assess the performance of machine learning predictions, primarily due to the challenges posed by imbalanced data.

In this section, we also summarize and analyze the features' importance in mini flash crash predictions; the histograms of all windows' features importance are provided in Internet Appendix 8.

5.1 Nanex (2010) Mini Flash Crashes Prediction

5.1.1 Tuning and Testing

In this section, our focus is directed towards the real-time prediction of mini flash crashes that adhere to the strictest criteria outlined by Nanex (2010), requiring a time range not exceeding 1.5 seconds. As in the preceding sections, we explore scenarios both with and without reversals, employing various strategies for processing imbalanced data in conjunction with eight distinct machine learning models. In accord with our previous approach, we aggregate the counts of each feature selected by the reduced model when implementing reduced-feature models to evaluate the importance of individual features. Given the substantial data processing workload, we perform undersampling of the data without mini flash crashes to a scale 100 times the total number of mini flash crashes in each month within each rolling window, under the condition that the ratio of time intervals in all intervals is lower than 1% for each stock.

Table 3 and Table 4 present the AUC tuning results for all 1.5s Nanex (2010) mini flash crashes without reversals across all eight windows. As can be observed, ensemble undersampling achieves the highest performance across all eight windows for both versions, with and without reversals, of 1.5s mini flash crashes. For the versions without reversals, XGBoost combined with ensemble undersampling demonstrates the best performance in four windows, while reduced-feature XGBoost achieves the highest AUC in three of the eight windows. The remaining window sees the best performance from reduced-feature random forest, which is also an ensemble-style model. In comparison to the earlier sections, the tuning of the AUC results is not as successful, with five AUC values exceeding 0.8, including two exceeding 0.9. In the remaining three windows, the best AUC values are 0.774, 0.778, and 0.736. While not perfect, these values still indicate a reasonable level of predictability.

The tuning AUC values for Nanex (2010) mini flash crashes with reversals are presented in Table 5 and Table 6. Our results are generally consistent with the versions without

reversals. XGBoost combined with ensemble undersampling exhibits the best performance in four windows, while reduced-feature XGBoost achieves the highest AUC in three of the eight windows. The only window where random forest achieves the highest performance is Window 5, mirroring the results from the version without reversals. All the results, whether with or without reversals, are consistent with the expectation presented in Section 4, which highlights the prevalence of reversals in most mini flash crashes.

Utilizing the best parameter sets, models, and the optimal imbalanced data processing strategy of ensemble undersampling, the model performance was assessed. The Nanex (2010) definition necessitates a time range not exceeding 1.5 seconds, categorizing these mini flash crashes as extreme low-probability events that are more challenging to predict than any of our previous versions. Consequently, it is essential to set realistic expectations for the test results. Nevertheless, a commendable performance is still achieved.

For both 1.5s Nanex (2010) mini flash crashes with, and without, reversals, the AUC values from the out-of-sample tests indicate that five of eight windows exceeded 0.8. In the other windows (Windows 1, 7 and 8), the AUC falls within the range of 0.7 to 0.8. The results for 1.5s Nanex (2010) mini flash crashes without reversals are presented in the last row of Panel A in Table 9, while the reversal test results are displayed in the last row of Panel B.

Our findings provide evidence that mini flash crashes, as defined by Nanex (2010), are universally predictable by machine learning. This discovery sheds light on the development of an early warning system for mini flash crashes, offering investor protection and enhancing market stability ahead of circuit breakers.

5.1.2 Feature Importance

Relying on the successful prediction, we also evaluated the feature importance in 1.5s mini flash crashes. While the critical role played by similar features in prediction is observed, including those consistent with the following three versions of mini flash crash definitions,

such as *LobImbalance*, *Lambda*, *QuotedSpread*, and *ESpread*, it is significant that the substantial advantages of specific features like *LobImbalance* and *Lambda* have diminished. In certain windows of prediction, volume-related features such as *VolumeAll*, *VolumeAvg*, and *VolumeMax* have become equally, if not more, important. Moreover, in some windows, the role of real-time microstructure noise, calculated according to Li and Linton (2022), seems to have become more vital than in any of the previous versions.

Due to the stringent 1.5-second definition of mini flash crashes by Nanex (2010), predicting these extremely rare events poses a significant challenge. Feature importance statistics reveal that relying on individual or a few features as signals for mini flash crashes becomes less feasible in machine learning models. The impact of limit order book imbalance on driving mini flash crashes decreases, making it harder to rank the importance of multiple features. This suggests that these rigorously defined mini flash crashes, characterized by their low-probability nature, may stem from a complex interplay of real-time market conditions and various trading algorithms. While somewhat predictable, they defy easy summarization in terms of identifying regular patterns, akin to human recognition of images or scenes. Machine learning algorithms, leveraging extensive data processing, can identify these patterns to some extent, offering a degree of predictability for mini flash crashes.

We've observed that volume and trade-related features, such as Volume Average and Breadth, strongly influence prediction success. This is interpretable: as a mini flash crash nears, informed traders swiftly impact the market, with algorithms and HFTs detecting anomalies within seconds. This information manifests in quotes and transactions just before the crash. The significance of *ReMeDI* performance, assessing short-term microstructure noise, further confirms this observation.

Our analysis emphasizes the significant impact of microstructure measures like Kyle's lambda and the Probability of Informed Trading (VPIN) on prediction accuracy. Kyle's lambda represents the cost of acquiring liquidity within a timeframe, while VPIN estimates the presence of informed traders with superior information. These findings suggest that

informed traders likely play a crucial role in triggering mini flash crashes. They use private information to influence market liquidity, raising costs for other investors seeking liquidity and driving the market towards a crash.

5.2 Short Term Mini Flash Crashes Prediction

5.2.1 Tuning and Testing

In this section, we shift our focus to the real-time prediction of short-term (15s) mini flash crashes, both with and without reversals. We apply a combination of data imbalanced processing strategies and eight distinct machine learning models to determine the most effective approach. Given the brief 15-second duration of these mini flash crashes, it is reasonable to assume that they are predominantly driven by trading algorithms, even if these algorithms are not necessarily as fast as High-Frequency Trading (HFT) algorithms. Utilizing a methodology akin to that outlined in Section 5.1, we examine a dataset encompassing the period from January 2017 to December 2018. Across this 24-month span, the data is segmented into eight discrete windows, each spanning a three-month duration. Within each window, the initial training dataset comprises the first month, while the second month is dedicated to tasks such as hyper-parameter optimization, model selection, and identification of optimal data imbalanced processing strategies.

This section, along with subsequent sections, exclusively showcases the best-selected tuning model and its corresponding results. In Panel A of Table 7 and Table 8, we present the optimal AUC tuning results for all 15s mini flash crashes without reversals across the eight defined windows. The combination of ensemble undersampling and reduced-feature XGBoost consistently outperforms alternative approaches in six out of eight windows. Notably, ensemble undersampling in conjunction with reduced-feature random forest and reduced-feature XGBoost represents two other effective strategies. All tuning AUC scores either meet or surpass 0.8, with some even exceeding the 0.9 threshold. The consistent superiority of ensemble undersampling underscores its effectiveness in addressing data imbalance. This performance

provides further evidence that the time interval preceding mini flash crashes often exhibits robust patterns compared to normal intervals. While these patterns may pose challenges in terms of causal interpretation, machine learning proves adept at detecting them.

The tuning AUC values for 15s mini flash crashes with reversals are available in Panel B of Table 7 and Table 8. The combination of ensemble undersampling and XGBoost consistently excels in seven out of eight windows and in five out of nine windows. In the remaining three windows, reduced-feature random forest and full-feature XGBoost achieve outstanding performance, both being ensemble-style models akin to reduced-feature XGBoost. All AUC values exceed 0.87, with some exceeding 0.9, demonstrating strong predictive performance.

Adhering to the identical process, we proceed to retrain the models utilizing the finest parameter sets, selected models, and the optimal data imbalanced processing strategy identified during the tuning procedure. The training dataset aligns with the second month of each window, and we subsequently perform out-of-sample tests deploying the test data from the third month within each window. The outcomes for short-term (15s) mini flash crashes without reversals are delineated in the third row of Panel A in Table 9, whereas the results for reversal tests are exhibited in the third row of Panel B.

In both short-term mini flash crashes, both with and without reversals, the AUC values derived from out-of-sample tests showcase remarkably robust performance. Across 14 of 16 windows, the AUC exceeds 0.9, with the lowest recorded value standing at 0.88.

These findings emphasize the persistent strength of patterns in short-term (15s) mini flash crashes, primarily influenced by trading algorithms. This substantiates our anticipation of leveraging machine learning-based anomaly detection as an advanced model for establishing an early warning system to detect mini flash crashes before circuit breakers are triggered.

5.2.2 Feature Importance

The feature importance in short-term (15s) mini flash crashes displays variations comparable to the subsequent two versions. Although the limit order book imbalance stands out as one

of the strongest predictors, in specific windows variables such as *Lambda*, *QuotedSpread*, and *Turnover* outshine *LobImbalance*. This divergence is notable and contrasts with the patterns observed in subsequent sections covering long and medium-term analyses.

As defined in Section 3.1, *Lambda* measures the price change in the interval relative to the total volume, and *Turnover* represents the speed of transactions. Both variables provide insights into actual transactions, in addition to bid and ask quotes. This discovery may imply that in the case of short-term (15s) mini flash crashes primarily driven by trading algorithms, as these events approach, the triggering of specific stop-loss programs likely results in a significantly larger number of market orders being submitted by trading algorithms. This is done to rapidly liquidate or reposition assets. These actions, which impact trading volume within the market, may further contribute to the onset of mini flash crashes. These behaviors can be captured by turnover rate and lambda, making them important signals in predicting 15s mini flash crashes.

Furthermore, volume-related features such as Volume Average and Breadth continue to exhibit their strength. Additionally these, microstructure measures such as Kyle's lambda, the Probability of Informed Trading (VPIN), and *RollMeasure* demonstrate stronger performance than in the long and medium-term versions of the empirical results. This discovery may suggest that in even shorter and faster mini flash crash events, informed trading might play an even more pivotal role. Importantly, all our findings for mini flash crashes, whether with or without reversals, are consistent

5.3 Medium Term Mini flash crashes Prediction

5.3.1 Tuning and Testing

In this section, we shift our focus to the real-time prediction of medium-term (90s) mini flash crashes, both with and without reversals. We utilize the same combination of imbalanced data processing strategies as employed in the previous section on Nanex (2010) version (1.5s) and medium-term (15s) mini flash crashes. The primary distinction from the prior

two versions lies in the 90s timeframe, where slow traders may contribute to mini flash crashes, not just fast traders as observed in algorithmic scenarios.

As with the earlier sections covering 1.5s and 15s, we analyze a dataset spanning two years, from January 2017 to December 2018. Over this 24-month period, we segment the data into eight distinct windows, each spanning a three-month period. To capture the most recent information within each window, the first month serves as the initial training dataset, and the second month is dedicated to hyper-parameter optimization, model selection, and the identification of optimal imbalanced data processing strategies.

The Panel A in Table 7 and Table 8 presents the best AUC tuning results for all 90s mini flash crashes without reversals across all eight windows. The combination of ensemble undersampling and reduced-feature XGBoost consistently outperforms other approaches in six of all eight windows. All tuning AUC scores reach or exceed 0.8, with some approaching 0.9. For the other two windows, direct undersampling shows the best performance through reduced-features random forest and XGBoost in Windows 7 and 8, respectively. These findings provide evidence that mini flash crashes in the medium term also have strong patterns that could be detected in real time by machine learning.

The tuning AUC values for 90s mini flash crashes with reversals are available in Panel B of Table 7 and Table 8. The combination of ensemble undersampling and reduced-feature XGBoost consistently excels in seven of eight windows. The only exception is Window 1, covering data from July 2017 to September 2017, where XGBoost, in conjunction with ensemble undersampling, achieves the highest performance with an AUC of 0.96. As with mini flash crashes without strict requirements on reversals, machine learning models demonstrate strong predictive performance.

Having utilized the best parameter sets, models, and the optimal imbalanced data processing strategy of ensemble undersampling, we retrain the models using the tuning data, which corresponds to the second month of each window. Subsequently, we conduct out-of-sample tests using the test data, representing the third month in each window. The results

for medium-term (90s) mini flash crashes without reversals are presented in the second row of Panel A in Table 9, while the reversal test results are displayed in the second row of Panel B.

For both medium-term mini flash crashes with and without reversals, the AUC values from the out-of-sample tests indicate remarkably strong performance. In 14 of 16 windows, the AUC exceeds 0.9, with the lowest value being 0.892. These results demonstrate the robust predictability of mini flash crashes at the minute level.

5.3.2 Feature Importance

The feature importance in medium-term mini flash crashes exhibits a striking consistency with the following long-term version. The feature with the highest importance is the limit order book imbalance, reinforcing our finding that, on a minute-to-minute level, mini flash crashes are heavily influenced by information asymmetry.

Additionally, variables such as *Lambda*, *QuotedSpread*, and *ESpread* continue to demonstrate their significant importance in predicting mini flash crashes. These variables reflect short-term market liquidity and stability, maintaining a robust correlation with the incidence of mini flash crashes across all eight window configurations.

Furthermore, the strong performance of Volume Average and Breadth provides further evidence that informed traders with privileged information can swiftly influence the market. Algorithms and HFTs can rapidly detect anomalies, resulting in increased trading activity and liquidity imbalances just moments before an actual crash occurs. This observation is further supported by the noteworthy performance of *ReMeDI* as a feature, which assesses short-term microstructure noise.

Our analysis also emphasizes the substantial influence of microstructure measures, such as Kyle's lambda and the Probability of Informed Trading (VPIN), on prediction accuracy

5.4 Long Term Mini flash crashes Prediction

5.4.1 Tuning and Testing

In this section, we explore real-time predictions of long-term (180s) mini flash crashes, covering scenarios with and without reversals. The key distinction between these two definitions lies in the assumption that within a 180s timeframe, it is reasonable to presume that even slow traders, such as humans, can exert influence. The market has the capacity to assimilate information from both fast traders, such as HFTs, and slow traders. However, in a 90s timeframe, slow traders may only contribute “a portion of their information” to the market.

We employ all five strategies for processing imbalanced data alongside the original dataset, which serves as our baseline. The objective is to assess eight distinct machine learning models to identify the best-performing model. These models encompass Logistic regression with l_1 regularization, support vector machines, random forests, and XGBoost, each with its corresponding reduced-feature model. Reduced-feature models are generated by excluding features that do not significantly contribute to predictability. Furthermore, when implementing reduced-feature models, we aggregate the counts of each selected feature to evaluate the importance of individual features.

To ensure a comprehensive understanding of long-term trends and the robustness of our findings, we scrutinize a dataset spanning two years, from January 2017 to December 2018. Over this 24-month duration, we partition the data into eight distinct windows, each encompassing a three-month period. To capture the most recent information, within each window, the first month is allocated for the initial training dataset, the second month is dedicated to hyper-parameter optimization, model selection, and the identification of the optimal imbalanced data processing strategies.

The Panel A in Table 7 and Table 8 present the best AUC tuning results for all 180s mini flash crashes without reversals across all eight windows. It is evident that the combination of ensemble undersampling and reduced-feature XGBoost consistently delivers the best

performance in seven out of eight windows. Window 1, is very close to being the best, with the direct undersampling approach taking the top spot. All our tuning AUC scores achieve values of 0.8 or above, with some even approaching 0.9.

Furthermore, it is worth noting that while direct undersampling may lead to a substantial loss of data, it still manages to achieve reasonably good performance. This observation suggests that the signals associated with "mini flashes" can be quite robust when compared to the time intervals preceding mini flash crashes. Through the ensemble undersampling approach, which involves comparing all time intervals marked with mini flash crashes and systematically reconstructing a balanced dataset by integrating them with other datasets containing no mini flash crashes, we have successfully achieved the best performance.

The tuning AUC values for 180s mini flash crashes with reversals are available in Panel B of Table 7 and Table 8. The combination of ensemble undersampling and reduced-feature XGBoost consistently delivers the best performance across seven of the eight windows. The sole exception is Window 3, covering data from July 2017 to September 2017, where XGBoost outperforms and achieves an AUC of 0.86. As with mini flash crashes without strict requirements on reversal, the machine learning models exhibit strong predictive performance. This observation clearly highlights the distinct patterns in the limit order book when a mini flash crash is impending.

By employing the best parameter sets, models, and the optimal imbalanced data processing strategy of ensemble undersampling, we retrain the models using the tuning data, which corresponds to the second month of each window. Subsequently, we conduct out-of-sample tests using the test data, which represents the third month in each window. The results for long-term (180s) mini flash crashes without reversal are presented in the first row of Panel A in Table 9, while the reversal test results are displayed in the first row of Panel B.

For both long-term mini flash crashes with and without reversals, the AUC values from the out-of-sample tests indicate remarkably strong performance. In 14 of 16 windows, the AUC exceeds 0.9, with the lowest value being 0.899.

Furthermore, as evident from the data description and pre-processing section, most mini flash crashes exhibit reversals. This observation leads to a reasonable inference that mini flash crashes are events intricately linked to market conditions and are significantly influenced by the stability of the financial market.

5.4.2 Feature Importance

It is noteworthy that through all eight windows and for both long term mini flash crashes with and without reversal, the limit order book imbalance takes the first place on feature importance. The limit order book imbalance encompasses the differences in stock market information that investors have regarding price changes at a given moment, as reflected by disparities in the quotes provided by buyers and sellers. This market condition is highly likely to be strongly correlated with the occurrence of mini flash crashes. This finding is highly reasonable as it highlights how short-term market information asymmetry could be a contributing factor to mini flash crashes. In fact, high-speed trading algorithms may exacerbate this phenomenon within the trading ecosystem.

In addition to the limit order book imbalance, variables such as *Lambda*, *QuotedSpread*, and *ESpread* also exhibit significant importance in predicting mini flash crashes. They all reflect the short-term market liquidity and stability and show a strong correlation with the occurrence of mini flash crashes. This consistent result holds across all eight window configurations, both with and without reversal, in our predictive analysis.

Based on the discussion of utilizing machine learning to predict mini flash crashes under the four different definitional models, we empirically demonstrate that, given real-time limit order book data, microstructure features exhibit identifiable patterns when a mini flash crash is imminent. However, despite this, it is not wise to construct arbitrage strategies based on these predictions, as market liquidity rapidly depletes during mini flash crashes. Instead, regulators, exchanges, and mutual funds can use our findings to develop an early warning system to alert for mini flash crashes.

6 Economics and Policy Implication

Our research using machine learning to predict mini flash crashes through market microstructure features includes Kyle Lambda, bid-ask spread, and limit order book imbalance with key theoretical models and provides valuable insights into the role of information asymmetry in financial markets.

According to the Glosten and Milgrom (1985), information asymmetry causes market makers to widen the bid-ask spread as a safeguard against the risk of trading with informed traders who possess superior information. The bid-ask spread s is given by:

$$s = \frac{2\kappa}{\lambda}$$

where κ represents the risk premium for adverse selection and λ is the precision of the public signal. As the risk premium κ increases or the precision λ decreases, the bid-ask spread widens.

This widening of the spread reflects increased adverse selection risks and signals potential liquidity issues, which can lead to mini flash crashes when these risks are realized.

The Kyle (1985) model further explains that informed traders, who trade based on private information, reduce market depth as they absorb liquidity from the market. In this model, the price P adjusts according to the order flow X through a linear relationship:

$$P = \alpha + \beta X$$

where α is the base price level and β represents market depth. Kyle lambda λ is defined as:

$$\lambda = \frac{1}{\beta}$$

A decrease in market depth indicates increased informed trading and suggests that the

market is more vulnerable to shocks, which can precipitate mini flash crashes if liquidity is rapidly depleted.

Our machine learning models effectively identify that mini flash crashes can be predicted by monitoring changes in market depth, Kyle lambda, and bid-ask spread. This empirical evidence supports the theoretical perspectives that, despite the sophistication and complexity of high-frequency trading algorithms, information asymmetry and the informational advantages of informed traders remain significant drivers of market liquidity.

Our findings further validate the conclusions of Easley et al. (2021), who demonstrate how machine learning can be effectively applied to microstructure research. In response to their observation that microstructure-based measures are useful for out-of-sample prediction of various market statistics, we provide additional insights by suggesting that this predictive power may be attributed to the strategic trading algorithms employed by informed traders, which exhibit discernible patterns.

Furthermore, we have found that the feature we defined as “breath,” which represents transaction volume, also provides significant predictive power in our model. This suggests that when a mini flash crash is imminent, high-frequency traders (HFTs) are likely to leverage their asymmetric information advantages to strategically consume market liquidity. This strategic behavior results in a notable impact on transactions, as well as other features related to Kyle lambda and trading volume. Together, these features signal emerging patterns indicative of an imminent mini-flash crash.

Thus, even in the era of high-frequency trading, where trading behaviors are highly complex and algorithm-driven, the fundamental issue of information asymmetry continues to play a crucial role in causing mini flash crashes. This highlights the need for targeted regulatory measures to address information asymmetry and enhance market stability.

To mitigate the impact of HFT and its ability to exploit information asymmetry, several solutions can be considered. An approach is the implementation of speed bumps, which are temporary delays introduced in the trading process to level the playing field between high-

frequency and traditional traders. Speed bumps can reduce the advantages of ultra-fast trading algorithms by introducing a short, uniform latency for all trades, thereby mitigating the impact of HFT on market liquidity and volatility.

Another solution involves minimum resting times for orders, which require trades to remain in the order book for a minimum period before being canceled. This measure can help prevent predatory practices, such as quote stuffing, and reduce the frequency of rapid, manipulative trades.

In addition, order-to-trade ratios can be imposed, limiting the number of orders that can be placed relative to the number of trades executed. This helps curb excessive order placement and cancellation, which can lead to market manipulation and increased volatility.

Increased transparency and reporting requirements for algorithmic trading strategies may also be beneficial. By mandating detailed disclosures of trading algorithms and their strategies, regulators can better monitor and understand the behaviors of high-frequency traders, allowing for more informed oversight and intervention.

Finally, market-making obligations for HFT firms can be established to ensure that these traders contribute positively to market liquidity and stability. By requiring HFT firms to maintain a certain level of market making, regulators can ensure that these firms play a constructive role in the market rather than solely focusing on exploiting information asymmetry for short-term gains.

These regulatory measures can help address the challenges posed by high-frequency trading and improve overall market stability by reducing the advantages of information asymmetry and ensuring fairer trading practices.

7 Conclusion

In this study, we pioneer the application of machine learning techniques to predict real-time mini flash crashes in individual stocks. Our research provides empirical evidence and methodologies for forecasting mini flash crashes using data from the limit order book across

four distinct time frames: 180 seconds, 90 seconds, 15 seconds, and 1.5 seconds. Notably, all these intervals fall within the 5-minute circuit breaker system set by the SEC, suggesting the viability of real-time early warning systems for mini flash crashes. Our analysis spans mini flash crashes in S&P100 stocks from 2017 to 2018, revealing that most of these events swiftly reverse within a short time frame. This observation underscores the systemic issues in current market mechanisms, particularly the prevalence of high-frequency algorithmic trading, as the primary drivers of mini flash crashes.

We introduce different time frames based on the involvement of slow traders and fast, automated trading algorithms. The 180-second and 90-second time frames accommodate the participation of slow traders, contributing to enhanced predictability, while the 15-second and 1.5-second time frames focus more on algorithmic trading behaviors. Our findings indicate varying levels of predictability across these time frames, with machine learning models showing efficacy in forecasting mini flash crashes, albeit with decreasing predictive accuracy in ultra-short duration. Overall, our research underscores the importance of monitoring real-time market dynamics and offers insights for investors and regulatory authorities in effectively managing and mitigating the impact of mini flash crashes.

References

- Andersen, T. G. and O. Bondarenko (2014a). Assessing Measures of Order Flow Toxicity and Early Warning Signals for Market Turbulence. *Review of Finance* 19(1), 1–54.
- Andersen, T. G. and O. Bondarenko (2014b). Reflecting on the vpin dispute. *Journal of Financial Markets* 17, 53–64.
- Aït-Sahalia, Y., J. Fan, L. Xue, and Y. Zhou (2022). How and when are high-frequency stock returns predictable? Technical report, National Bureau of Economic Research, Inc.
- Baron, M., J. Brogaard, B. Hagströmer, and A. Kirilenko (2019). Risk and return in high-frequency trading. *The Journal of Financial and Quantitative Analysis* 54(3), 993–1024.
- Bellia, Mario; Christensen, K. K. A. P. L. R. R. (2020). High-frequency trading during flash crashes: Walk of fame or hall of shame. *SAFE Working Paper No. 270.*, 68.
- Bossaerts, P., C. Frydman, and J. Ledyard (2013). The Speed of Information Revelation and Eventual Price Quality in Markets with Insiders: Comparing Two Theories. *Review of Finance* 18(1), 1–22.
- Brogaard, J., A. Carrion, T. Moyaert, R. Riordan, A. Shkilko, and K. Sokolov (2018). High frequency trading and extreme price movements. *Journal of Financial Economics* 128(2), 253–265.
- Chiu, J. and C.-H. Chen (2023). Limit order revisions across investor sophistication. *Journal of Empirical Finance* 70, 74–90.
- Cont, R., A. Kukanov, and S. Stoikov (2013). The price impact of order book events. *Journal of Financial Econometrics* 12(1), 47–88.
- Deng, X., S. Hung, and Z. Qiao (2018). Mutual fund herding and stock price crashes. *Journal of Banking & Finance* 94(C), 166–184.

- Dugast, J. and T. Foucault (2018). Data abundance and asset price informativeness. *Journal of Financial Economics* 130(2), 367–391.
- Dyakov, T. and M. Verbeek (2013). Front-running of mutual fund fire-sales. *Journal of Banking & Finance* 37(12), 4931–4942.
- Easley, D. (2013). *High-frequency Trading: New Realities for Traders, Markets and Regulators*. Risk Books.
- Easley, D., M. L. de Prado, M. O’Hara, Z. Zhang, and W. Jiang (2021). Microstructure in the machine age. *Review of Financial Studies* 34(7), 3316–3363.
- Easley, D., M. Lopez de Prado, and M. O’Hara (2012). Flow toxicity and liquidity in a high frequency world. *Review of Financial Studies* 25(5), 1457–1493.
- Fosset, A., J.-P. Bouchaud, and M. Benzaquen (2023). Non-parametric estimation of quadratic hawkes processes for order book events. *Quantitative Finance* 23(5), 741–758.
- Foucault, T. (2016). Where are the risks in high frequency trading? *Financial Stability Review*, 53–67.
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics* 29(5), 1189–1232.
- Glosten, L. R. and P. R. Milgrom (1985). Bid, ask and transaction prices in a specialist market with heterogeneously informed traders. *Journal of Financial Economics* 14(1), 71–100.
- Golub, A., J. Keane, and S.-H. Poon (2012). High frequency trading and mini flash crashes. *Available at: SSRN 2182097*.
- Hastie, T., R. Tibshirani, and J. Friedman (2009). *The elements of statistical learning: data mining, inference and prediction* (2 ed.). Springer.

- Hendershott, T. and P. C. Moulton (2011). Automation, speed, and stock market quality: The nyse's hybrid. *Journal of Financial Markets* 14(4), 568–604.
- Keller, A. J. (2012). Regulating high frequency trading after the flash crash of 2010. *Ohio State Law Journal* 73(6), 1457–1483.
- Kercheval, A. N. and Y. Zhang (2015). Modelling high-frequency limit order book dynamics with support vector machines. *Quantitative Finance* 15(8), 1315–1329.
- Kirilenko, A., A. S. Kyle, M. Samadi, and T. Tuzun (2017). The flash crash: High-frequency trading in an electronic market. *The Journal of Finance* 72(3), 967–998.
- Kyle, A. S. (1985). Continuous auctions and insider trading. *Econometrica* 53(6), 1315–1335.
- Leal, S. J., M. Napoletano, A. Roventini, and G. Fagiolo (2014). Rock around the clock: An agent-based model of low- and high-frequency trading. *GREDEG Working Papers* 2014-21.
- Lee, C. and M. Ready (1991). Inferring trade direction from intraday data. *Journal of Finance* 46(2), 733–46.
- Levine, M. (2015). Guy trading at home caused the flash crash. *Bloomberg*.
- Li, M., X. Yin, and J. Zhao (2020). Does program trading contribute to excess comovement of stock returns? *Journal of Empirical Finance* 59, 257–277.
- Li, Z. M. and O. Linton (2022). A ReMeDI for Microstructure Noise. *Econometrica* 90(1), 367–389.
- Mase, B. (1999). The Predictability of Short-Horizon Stock Returns. *Review of Finance* 3(2), 161–173.

- Menkhoff, L., C. L. Osler, and M. Schmeling (2010). Limit-order submission strategies under asymmetric information. *Journal of Banking & Finance* 34(11), 2665–2677.
- Mohajon, J. (2020). Confusion matrix for your multi-class machine learning model. *towards-datascience*.
- Murphy, K. P. (2013). *Machine learning : a probabilistic perspective*. Cambridge, Mass. The U.S.A: MIT Press.
- Nanex (2010). Flash equity failures in 2006, 2007, 2008, 2009, 2010, and 2011. *Available at: www.nanex.net*.
- Nolte, D. (2010). SEC Takes Initial Measures to Avoid a Second Flash Crash. *Available at: HGExperts.com*.
- OICU-IOSCO (2011). Regulatory issues raised by the impact of technological changes on market integrity and efficiency consultation report. *Available at: https://www.iosco.org*.
- O’Hara, M. (2015). High frequency market microstructure. *Journal of Financial Economics* 116(2), 257–270.
- Rad, H., R. K. Y. Low, J. Miffre, and R. Faff (2023). The commodity risk premium and neural networks. *Journal of Empirical Finance* 74, 101433.
- Scholtus, M., D. van Dijk, and B. Frijns (2014). Speed, algorithmic trading, and market quality around macroeconomic news announcements. *Journal of Banking & Finance* 38(C), 89–105.
- Wang, X., M. H. Kim, and S. Suardi (2022). Herding and china’s market-wide circuit breaker. *Journal of Banking & Finance* 141(C).
- Watanabe, T. and J. Nakajima (2024). High-frequency realized stochastic volatility model. *Journal of Empirical Finance* 79, 101559.

Zhang, L., P. A. Mykland, and Y. Aït-Sahalia (2005). A tale of two time scales. *Journal of the American Statistical Association* 100(472), 1394–1411.

Zhu, H., L. Bai, L. He, and Z. Liu (2023). Forecasting realized volatility with machine learning: Panel data perspective. *Journal of Empirical Finance* 73, 251–271.



Source: Levine (2015)

Figure 1: Flash Crash, 06 May 2010

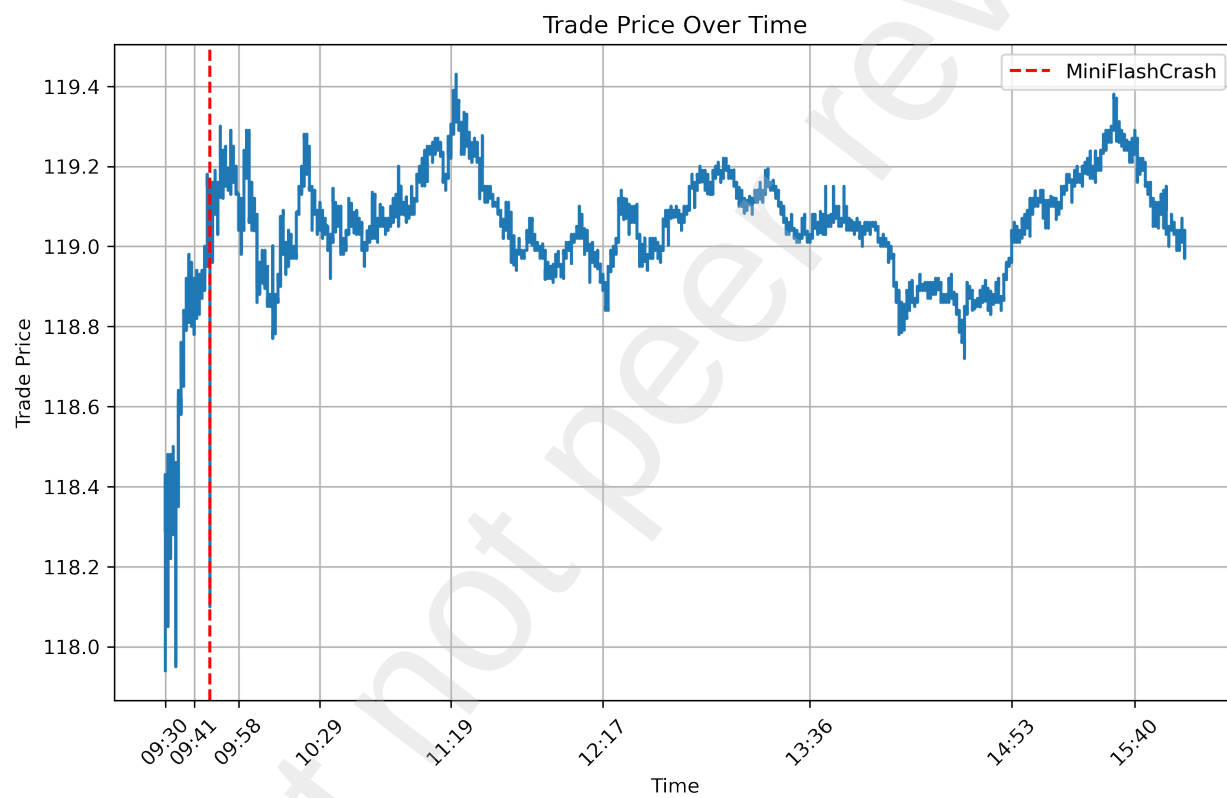
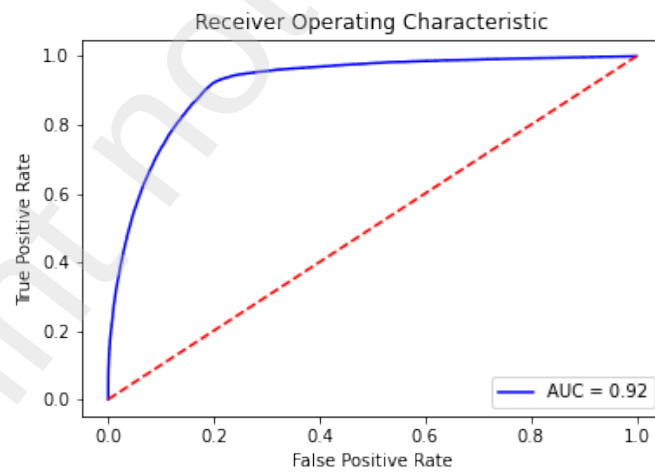


Figure 2: Mini flash crash, Nanex (2010) definition, AAPL, 01/09/2017

		True Class	
		Positive	Negative
Predicted Class	Positive	TP	FP
	Negative	FN	TN

Source: Mohajon (2020)

Figure 3: Confusion Matrix



ROC of our best tuning result of 15s Mini Flash Crash in Window 7

Figure 4: ROC

Window1					Window2					Window3					Window4				
Train		180s	90s	15s	1.5s	180s	90s	15s	1.5s	180s	90s	15s	1.5s	180s	90s	15s	1.5s		
Train																			
No. Stocks:		92	91	54	19	81	77	33	14	82	70	28	8	89	82	41	22		
0:		222,953	462,021	2,993,543	30,314,733	215,398	444,430	2,849,597	28,769,886	244,541	497,331	3,031,001	30,323,834	255,366	521,831	3,243,399	32,601,218		
1:		29,747	43,379	38,857	9,267	24,401	35,168	27,991	5,994	8,159	8,069	1,399	166	16,353	21,607	17,229	5,062		
Ratio:		13.34%	9.39%	1.30%	0.03%	11.33%	7.91%	0.98%	0.02%	3.37%	1.62%	0.05%	5×10 ⁻⁴ %	6.40%	4.14%	0.53%	0.02%		
Tune																			
0:		213,814	442,553	2,850,253	28,801,256	251,140	519,132	3,302,603	33,316,820	282,267	573,961	3,486,235	34,872,484	239,902	495,788	3,084,512	30,971,492		
1:		26,251	37,577	30,527	6,544	26,564	36,276	29,845	7,660	8,338	7,249	1,025	116	18,251	20,518	13,324	6,868		
Ratio:		12.27%	8.49%	1.07%	0.02%	10.58%	6.99%	0.90%	0.02%	2.95%	1.26%	0.03%	3.3×10 ⁻⁴ %	7.61%	4.14%	0.43%	0.02%		
Test																			
0:		259,790	537,320	3,449,582	34,863,892	267,022	544,736	3,333,017	33,356,129	224,450	463,547	2,951,403	29,772,659	227,719	472,972	2,966,048	29,806,386		
1:		30,815	43,890	37,678	8,708	10,948	11,204	2,623	271	23,728	32,809	26,733	8,701	20,725	23,916	15,280	6,894		
Ratio:		11.86%	8.17%	1.09%	0.02%	4.10%	2.06%	0.08%	8.1×10 ⁻⁴ %	10.57%	7.08%	0.91%	0.03%	9.10%	5.06%	0.51%	0.02%		
Window5																			
Train		180s	90s	15s	1.5s	180s	90s	15s	1.5s	180s	90s	15s	1.5s	180s	90s	15s	1.5s		
Window6																			
No. Stocks:		93	93	59	34	90	88	46	21	88	84	52	19	92	92	84	45		
0:		240,882	499,205	3,135,355	31,483,995	243,635	503,211	3,138,680	31,455,558	241,451	499,866	3,126,088	31,475,602	257,666	536,657	3,400,811	34,308,989		
1:		21,527	25,613	13,553	5,085	18,508	21,075	7,036	1,602	20,958	24,952	22,820	13,478	28,417	35,509	32,185	20,971		
Ratio:		8.94%	5.13%	0.43%	0.02%	7.60%	4.19%	0.22%	5×10 ⁻³ %	8.68%	4.99%	0.73%	0.04%	11.03%	6.62%	0.95%	0.06%		
Tune																			
0:		215,448	449,302	2,836,701	28,500,849	258,590	532,319	3,291,900	32,986,388	256,010	528,623	3,269,617	32,896,930	237,957	494,377	3,100,068	31,176,774		
1:		22,090	25,774	13,755	3,711	16,321	17,503	7,032	2,932	18,236	19,869	21,335	12,590	21,925	25,387	18,516	9,066		
Ratio:		10.25%	5.74%	0.48%	0.01%	6.31%	3.29%	0.21%	8.89×10 ⁻³ %	7.12%	3.76%	0.65%	0.04%	9.21%	5.14%	0.60%	0.03%		
Test																			
0:		244,624	505,052	3,145,115	31,504,509	243,618	502,663	3,125,179	31,442,651	218,135	451,570	2,805,565	28,177,196	207,544	432,827	2,785,092	28,186,570		
1:		17,918	20,032	5,389	531	18,525	21,623	20,537	14,509	16,743	18,186	12,971	8,164	27,467	37,195	35,040	14,750		
Ratio:		7.32%	3.97%	0.17%	1.6×10 ⁻³ %	7.60%	4.30%	0.66%	0.05%	7.68%	4.03%	0.46%	0.03%	13.23%	8.59%	1.26%	0.05%		

Table 1: Sample descriptive statistics of mini flash crashes without reversal

Window1				Window2				Window3				Window4			
180s	90s	15s	1.5s	180s	90s	15s	1.5s	180s	90s	15s	1.5s	180s	90s	15s	1.5s
Train															
No. Stocks:	92	89	44	18	75	30	11	81	64	25	7	88	81	39	20
0:	223,189	464,536	3,003,712	30,318,614	215,626	446,586	2,857,338	28,772,622	244,937	499,122	3,031,453	30,323,885	255,640	523,291	3,248,093
1:	29,511	40,864	28,688	5,386	24,173	33,012	20,250	3,258	7,763	6,278	947	115	16,079	20,147	12,535
Ratio:	13.22%	8.80%	0.96%	0.02%	11.21%	7.39%	0.71%	0.01%	3.17%	1.26%	0.03%	3.79 × 10 ⁻⁴	6.29%	3.85%	0.39%
Tune															
0:	214,074	444,661	2,858,674	28,804,205	251,541	521,712	3,310,816	33,320,420	282,812	575,715	3,486,595	34,872,520	241,053	499,114	3,088,258
1:	25,991	35,469	22,106	3,595	26,163	33,696	21,632	4,060	7,793	5,495	665	80	17,100	17,192	9,578
Ratio:	12.14%	7.98%	0.77%	0.01%	10.40%	6.46%	0.65%	0.01%	2.76%	0.95%	0.02%	2.29 × 10 ⁻⁴	7.09%	3.44%	0.31%
Test															
0:	260,140	539,818	3,459,361	34,867,804	267,333	546,502	3,333,796	33,356,199	224,712	465,722	2,958,328	29,776,581	228,885	476,292	2,970,256
1:	30,465	41,392	27,899	4,796	10,637	9,438	1,844	201	23,466	30,634	19,808	4,779	19,559	20,596	11,072
Ratio:	11.71%	7.67%	0.81%	0.014%	3.98%	1.73%	0.06%	6.03 × 10 ⁻⁴	10.44%	6.58%	0.67%	0.02%	8.55%	4.32%	0.37%
Window5															
Window5				Window6				Window7				Window8			
180s	90s	15s	1.5s	180s	90s	15s	1.5s	180s	90s	15s	1.5s	180s	90s	15s	1.5s
Train															
No. Stocks:	93	93	52	26	84	42	17	86	79	48	13	92	92	79	39
0:	241,647	502,257	3,139,262	31,486,054	244,155	505,747	3,141,160	31,456,084	242,546	503,207	3,131,021	31,480,601	258,217	539,712	3,406,986
1:	20,762	22,561	9,646	3,026	17,988	18,539	4,556	1,076	19,863	21,611	17,887	8,479	27,866	32,454	26,010
Ratio:	8.59%	4.49%	0.31%	9.61 × 10 ⁻³	7.37%	3.67%	0.15%	3.42 × 10 ⁻³	8.19%	4.29%	0.57%	0.03%	10.79%	6.01%	0.76%
Tune															
0:	215,964	451,659	2,840,512	28,501,890	259,403	535,035	3,293,622	32,987,443	257,264	531,410	3,274,296	32,901,994	238,458	497,202	3,104,191
1:	21,574	23,417	9,944	2,670	15,508	14,787	5,310	1,877	16,982	17,082	16,656	7,526	21,424	22,562	14,393
Ratio:	9.99%	5.18%	0.35%	9.37 × 10 ⁻³	5.98%	2.76%	0.16%	5.69 × 10 ⁻³	6.60%	3.21%	0.51%	0.02%	8.98%	4.54%	0.46%
Test															
0:	245,235	507,645	3,147,302	31,504,664	244,508	505,569	3,129,221	31,447,848	219,248	454,343	2,808,154	28,180,138	207,676	434,685	2,793,725
1:	17,307	17,439	3,202	376	17,635	18,717	16,495	9,312	15,630	15,413	10,382	5,222	27,335	35,337	26,407
Ratio:	7.06%	3.44%	0.10%	1.2 × 10 ⁻³	7.21%	3.70%	0.53%	0.03%	7.13%	3.39%	0.37%	0.02%	13.16%	8.13%	0.95%

Table 2: Sample descriptive statistics of mini flash crashes with reversal

Panel A	Window 1								Window 2							
	TM				OR				UD				TM			
	TM	OR	UD	OV	EN	SM	TM	OR	UD	OV	EN	SM	TM	OR	UD	OV
LOG	0.739	0.724	0.737	0.739	0.672	0.728	0.742	0.729	0.740	0.742	0.675	0.733	0.742	0.729	0.740	0.742
SVM	0.664	0.505	0.664	0.664	0.669	0.662	0.668	0.504	0.664	0.667	0.668	0.658	0.668	0.504	0.664	0.667
RF	0.690	0.690	0.763	0.695	0.748	0.679	0.595	0.657	0.757	0.624	0.691	0.713	0.595	0.657	0.757	0.624
XGB	0.727	0.744	0.748	0.724	0.774	0.662	0.660	0.715	0.747	0.661	0.777	0.660	0.660	0.715	0.747	0.661
RD&LOG	0.736	0.716	0.734	0.736	0.670	0.668	0.740	0.727	0.739	0.740	0.675	0.663	0.740	0.727	0.739	0.740
RD&SVM	0.665	0.505	0.665	0.665	0.667	0.663	0.666	0.504	0.663	0.664	0.669	0.655	0.666	0.504	0.663	0.664
RD&RF	0.688	0.683	0.766	0.700	0.769	0.676	0.595	0.673	0.754	0.610	0.765	0.691	0.595	0.673	0.754	0.610
RD&XGB	0.723	0.748	0.749	0.734	0.772	0.628	0.684	0.718	0.753	0.684	0.778	0.610	0.684	0.718	0.753	0.684

Panel B	Window 3								Window 4							
	TM				OR				UD				TM			
	TM	OR	UD	OV	EN	SM	TM	OR	UD	OV	EN	SM	TM	OR	UD	OV
LOG	0.466	0.539	0.717	0.471	0.731	0.482	0.733	0.699	0.769	0.747	0.687	0.749	0.733	0.699	0.769	0.747
SVM	0.637	0.572	0.732	0.637	0.784	0.627	0.688	0.507	0.690	0.687	0.689	0.689	0.688	0.507	0.690	0.687
RF	0.803	0.798	0.836	0.791	0.821	0.804	0.721	0.728	0.810	0.736	0.795	0.757	0.721	0.728	0.810	0.736
XGB	0.791	0.824	0.827	0.787	0.842	0.739	0.750	0.789	0.800	0.761	0.820	0.651	0.750	0.789	0.800	0.761
RD&LOG	0.467	0.534	0.752	0.472	0.743	0.628	0.725	0.684	0.760	0.728	0.674	0.672	0.725	0.684	0.760	0.728
RD&SVM	0.637	0.551	0.730	0.637	0.721	0.627	0.682	0.507	0.687	0.681	0.682	0.684	0.682	0.507	0.687	0.681
RD&RF	0.760	0.787	0.829	0.782	0.847	0.847	0.718	0.729	0.808	0.740	0.816	0.740	0.718	0.729	0.808	0.740
RD&XGB	0.770	0.794	0.838	0.811	0.850	0.796	0.766	0.796	0.801	0.748	0.820	0.619	0.766	0.796	0.801	0.748

Notes: This table presents the tuning results for mini flash crashes without reversal occurring within 1.5 seconds, spanning from window 1 to window 4 in the year 2017.

Table 3: Mini Flash Crash in 1.5s without reversal predication modeling tuning AUC, 2017

Panel A	Window 5								Window 6															
	TM				OR				UD				OV				EN				SM			
	TM	OR	UD	OV	EN	SM	TM	OR	UD	OV	EN	SM	TM	OR	UD	OV	EN	SM						
LOG	0.888	0.799	0.914	0.890	0.825	0.871	0.878	0.733	0.887	0.879	0.791	0.873	0.878	0.733	0.887	0.879	0.791	0.873	0.873					
SVM	0.570	0.579	0.740	0.573	0.764	0.560	0.705	0.503	0.791	0.705	0.788	0.688	0.705	0.503	0.791	0.705	0.788	0.688	0.688					
RF	0.877	0.927	0.958	0.915	0.948	0.869	0.861	0.925	0.931	0.886	0.912	0.909	0.861	0.925	0.931	0.886	0.912	0.909	0.909					
XGB	0.935	0.957	0.950	0.930	0.964	0.917	0.870	0.926	0.927	0.889	0.940	0.893	0.870	0.926	0.927	0.889	0.940	0.893	0.893					
RD&LOG	0.888	0.804	0.915	0.889	0.818	0.703	0.879	0.729	0.887	0.880	0.789	0.770	0.879	0.729	0.887	0.880	0.789	0.770	0.770					
RD&SVM	0.569	0.585	0.732	0.571	0.768	0.564	0.706	0.502	0.791	0.706	0.788	0.690	0.706	0.502	0.791	0.706	0.788	0.690	0.690					
RD&RF	0.903	0.941	0.956	0.916	0.965	0.814	0.866	0.922	0.925	0.884	0.932	0.886	0.866	0.922	0.925	0.884	0.932	0.886	0.886					
RD&XGB	0.929	0.956	0.946	0.926	0.963	0.907	0.897	0.926	0.928	0.882	0.938	0.886	0.897	0.926	0.928	0.882	0.938	0.886	0.886					

Panel B	Window 7								Window 8															
	TM				OR				UD				OV				EN				SM			
	TM	OR	UD	OV	EN	SM	TM	OR	UD	OV	EN	SM	TM	OR	UD	OV	EN	SM						
LOG	0.700	0.685	0.699	0.700	0.700	0.697	0.777	0.734	0.767	0.777	0.772	0.774	0.777	0.734	0.767	0.777	0.772	0.774	0.774					
SVM	0.642	0.501	0.640	0.642	0.641	0.639	0.670	0.508	0.688	0.669	0.691	0.670	0.670	0.508	0.688	0.669	0.691	0.670	0.670					
RF	0.686	0.684	0.724	0.698	0.723	0.691	0.814	0.817	0.846	0.825	0.845	0.715	0.814	0.817	0.846	0.825	0.845	0.715	0.715					
XGB	0.703	0.721	0.712	0.705	0.736	0.637	0.838	0.844	0.840	0.837	0.855	0.703	0.838	0.844	0.840	0.837	0.855	0.703	0.703					
RD&LOG	0.700	0.685	0.698	0.699	0.700	0.697	0.777	0.731	0.768	0.776	0.770	0.774	0.777	0.731	0.768	0.776	0.770	0.774	0.774					
RD&SVM	0.636	0.501	0.638	0.637	0.635	0.633	0.670	0.511	0.688	0.669	0.690	0.669	0.670	0.511	0.688	0.669	0.690	0.669	0.669					
RD&RF	0.688	0.683	0.722	0.700	0.730	0.686	0.814	0.817	0.845	0.823	0.851	0.591	0.814	0.817	0.845	0.823	0.851	0.591	0.591					
RD&XGB	0.716	0.726	0.714	0.713	0.736	0.607	0.841	0.846	0.840	0.840	0.854	0.699	0.841	0.846	0.840	0.840	0.854	0.699	0.699					

Notes: This table presents the tuning results for mini flash crashes without reversal occurring within 1.5 seconds, spanning from window 5 to window 8 in the year 2018.

Table 4: Mini Flash Crash in 1.5s without reversal predication modeling tuning AUC, 2018

Panel A	Window 1						Window 2					
	TM			OR			UD			OV		
	TM	OR	UD	OV	EN	SM	TM	OR	UD	OV	EN	SM
LOG	0.739	0.724	0.737	0.739	0.744	0.728	0.742	0.728	0.739	0.741	0.744	0.733
SVM	0.665	0.505	0.666	0.664	0.665	0.662	0.668	0.505	0.66	0.667	0.669	0.659
RF	0.685	0.69	0.762	0.695	0.748	0.679	0.605	0.669	0.752	0.608	0.687	0.714
XGB	0.725	0.746	0.748	0.725	0.774	0.651	0.663	0.721	0.749	0.671	0.777	0.638
RD&LOG	0.736	0.716	0.734	0.736	0.739	0.727	0.74	0.727	0.738	0.741	0.741	0.733
RD&SVM	0.665	0.505	0.668	0.665	0.665	0.663	0.665	0.504	0.658	0.664	0.669	0.656
RD&RF	0.688	0.684	0.761	0.7	0.771	0.675	0.597	0.671	0.756	0.621	0.768	0.691
RD&XGB	0.733	0.746	0.743	0.739	0.774	0.639	0.689	0.724	0.752	0.678	0.777	0.611

Panel B	Window 3						Window 4					
	TM			OR			UD			OV		
	TM	OR	UD	OV	EN	SM	TM	OR	UD	OV	EN	SM
LOG	0.518	0.484	0.689	0.527	0.733	0.548	0.729	0.698	0.756	0.744	0.75	0.749
SVM	0.634	0.614	0.659	0.613	0.745	0.647	0.685	0.507	0.684	0.686	0.688	0.688
RF	0.787	0.807	0.828	0.77	0.811	0.804	0.712	0.729	0.808	0.734	0.795	0.751
XGB	0.738	0.823	0.83	0.75	0.842	0.779	0.747	0.786	0.796	0.737	0.817	0.656
RD&LOG	0.518	0.485	0.711	0.526	0.731	0.545	0.721	0.681	0.744	0.728	0.744	0.728
RD&SVM	0.634	0.582	0.658	0.603	0.757	0.647	0.68	0.507	0.679	0.679	0.681	0.682
RD&RF	0.794	0.807	0.834	0.748	0.836	0.803	0.72	0.731	0.81	0.731	0.815	0.724
RD&XGB	0.734	0.804	0.816	0.779	0.83	0.768	0.758	0.789	0.797	0.771	0.820	0.639

This table presents the tuning results for mini flash crashes with reversal occurring within 1.5 seconds, spanning from window 1 to window 4 in the year 2017.

Table 5: Mini Flash Crash in 1.5s with reversal predication modeling tuning AUC, 2017

Panel A	Window 5						Window 6					
	TM	OR	UD	OV	EN	SM	TM	OR	UD	OV	EN	SM
LOG	0.881	0.786	0.912	0.881	0.918	0.867	0.877	0.731	0.848	0.878	0.893	0.872
SVM	0.564	0.573	0.762	0.565	0.751	0.555	0.695	0.503	0.764	0.696	0.798	0.678
RF	0.897	0.931	0.963	0.914	0.945	0.869	0.863	0.924	0.927	0.885	0.913	0.905
XGB	0.928	0.951	0.952	0.928	0.961	0.912	0.877	0.926	0.922	0.874	0.939	0.902
RD&LOG	0.881	0.786	0.916	0.888	0.906	0.879	0.878	0.731	0.851	0.879	0.892	0.874
RD&SVM	0.563	0.582	0.766	0.564	0.737	0.559	0.701	0.502	0.767	0.806	0.684	0.683
RD&RF	0.902	0.934	0.96	0.906	0.963	0.774	0.879	0.925	0.919	0.886	0.933	0.886
RD&XGB	0.933	0.957	0.956	0.932	0.962	0.899	0.875	0.925	0.916	0.889	0.936	0.891

Panel B	Window 7						Window 8					
	TM	OR	UD	OV	EN	SM	TM	OR	UD	OV	EN	SM
LOG	0.721	0.701	0.716	0.721	0.721	0.718	0.798	0.753	0.796	0.798	0.794	0.793
SVM	0.66	0.501	0.655	0.66	0.662	0.658	0.691	0.511	0.71	0.691	0.704	0.69
RF	0.695	0.695	0.742	0.705	0.742	0.712	0.808	0.824	0.858	0.822	0.856	0.741
XGB	0.718	0.737	0.728	0.722	0.757	0.646	0.843	0.855	0.848	0.839	0.867	0.733
RD&LOG	0.721	0.7	0.716	0.72	0.721	0.718	0.797	0.754	0.798	0.797	0.794	0.792
RD&SVM	0.655	0.501	0.652	0.654	0.655	0.652	0.692	0.513	0.711	0.692	0.707	0.691
RD&RF	0.697	0.699	0.744	0.708	0.751	0.704	0.809	0.826	0.859	0.824	0.865	0.627
RD&XGB	0.723	0.738	0.73	0.724	0.757	0.614	0.849	0.855	0.848	0.85	0.866	0.716

This table presents the tuning results for mini flash crashes with reversal occurring within 1.5 seconds, spanning from window 5 to window 8 in the year 2018.

Table 6: Mini Flash Crash in 1.5s with reversal predication modeling tuning AUC, 2018

Panel A: Mini Flash Crashes without reversal

Window1	Window2	Window3	Window4	Window5	Window6	Window7	Window8
180s: EN+RD&XGB	EN+RD&XGB	EN+RD&XGB	EN+RD&XGB	EN+RD&XGB	EN+RD&XGB	UD+RD&XGB	UD+RD&XGB
90s: EN+RD&XGB	EN+RD&XGB	EN+RD&XGB	EN+RD&XGB	EN+RD&XGB	EN+RD&XGB	EN+RD&RF	UD+RD&XGB
15s: EN+XGB	EN+XGB	EN+XGB	EN+XGB	EN+RD&RF	EN+RD&XGB	EN+XGB	EN+XGB
1.5s: EN+XGB	EN+RD&XGB	EN+RD&XGB	EN+RD&XGB	EN+RD&RF	EN+XGB	EN+XGB	EN+XGB

Panel B: Mini Flash Crashes with reversal

Window1	Window2	Window3	Window4	Window5	Window6	Window7	Window8
180s: EN+RD&XGB	EN+RD&XGB	EN+XGB	EN+RD&XGB	EN+RD&XGB	EN+RD&XGB	UD+RD&XGB	UD+RD&XGB
90s: EN+XGB	EN+RD&XGB	EN+RD&XGB	EN+RD&XGB	EN+RD&XGB	EN+RD&XGB	EN+RD&RF	UD+RD&XGB
15s: EN+XGB	EN+XGB	EN+RD&RF	EN+RD&RF	EN+EGB	EN+RD&XGB	EN+XGB	EN+XGB
1.5s: EN+RD&XGB	EN+RD&XGB	EN+XGB	EN+RD&XGB	EN+RD&RF	EN+XGB	EN+XGB	EN+XGB

Notes: This table presents the model with the best performance achieved by tuning its parameters and selecting an imbalanced data processing strategy. These choices were made using the training dataset, and the model's performance was subsequently assessed on the tuning data. "RD" denotes the reduced dimension model, in which only crucial features are selected.

Table 7: Best model combined imbalanced data strategy of Tuning

Panel A: Mini Flash Crashes without reversal

	Window1	Window2	Window3	Window4	Window5	Window6	Window7	Window8
180s:	0.895	0.895	0.864	0.829	0.850	0.877	0.872	0.812
90s:	0.896	0.897	0.856	0.853	0.849	0.885	0.878	0.829
15s:	0.856	0.877	0.812	0.934	0.878	0.918	0.919	0.925
1.5s:	0.774	0.778	0.850	0.820	0.965	0.940	0.736	0.855

Panel B: Mini Flash Crashes with reversal

	Window1	Window2	Window3	Window4	Window5	Window6	Window7	Window8
180s:	0.894	0.893	0.860	0.832	0.849	0.876	0.868	0.812
90s:	0.960	0.898	0.835	0.864	0.852	0.890	0.953	0.847
15s:	0.874	0.884	0.875	0.913	0.904	0.936	0.918	0.937
1.5s:	0.774	0.777	0.842	0.820	0.963	0.939	0.757	0.867

Notes: This table displays the Area Under the ROC Curve (AUC) values generated by the model chosen after optimizing its parameters and selecting an imbalanced data processing strategy. These selections were made using a tuning dataset, and the model's performance was subsequently evaluated on the test data.

Table 8: Best AUC of Tuning

Panel A: Mini Flash Crashes without reversal

	Window1	Window2	Window3	Window4	Window5	Window6	Window7	Window8
180s:	0.967	0.899	0.904	0.951	0.937	0.958	0.940	0.909
90s:	0.967	0.898	0.905	0.951	0.937	0.959	0.941	0.909
15s:	0.969	0.880	0.933	0.976	0.946	0.988	0.984	0.932
1.5s:	0.761	0.843	0.803	0.821	0.885	0.984	0.741	0.725

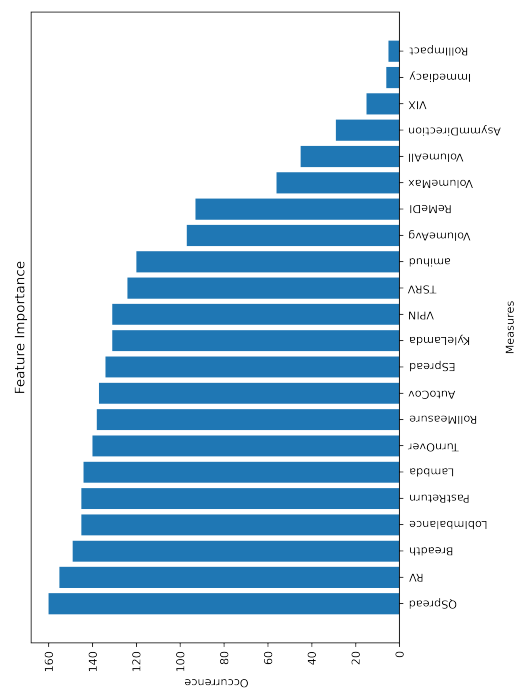
Panel B: Mini Flash Crashes with reversal

	Window1	Window2	Window3	Window4	Window5	Window6	Window7	Window8
180s:	0.964	0.904	0.915	0.943	0.926	0.956	0.938	0.900
90s:	0.967	0.892	0.903	0.954	0.942	0.958	0.944	0.908
15s:	0.967	0.895	0.945	0.979	0.940	0.990	0.987	0.932
1.5s:	0.769	0.868	0.817	0.816	0.877	0.984	0.740	0.714

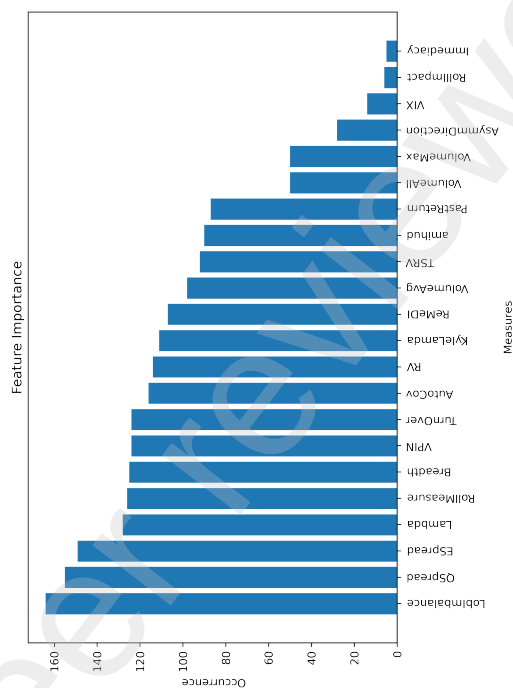
Notes: This table displays the Area Under the ROC Curve (AUC) values generated by the model chosen after optimizing its parameters and selecting an imbalanced data processing strategy. These selections were made using a tuning dataset, and the model's performance was subsequently evaluated on the test data.

Table 9: AUC of Test data set

8 Internet Appendix. Feature Importance

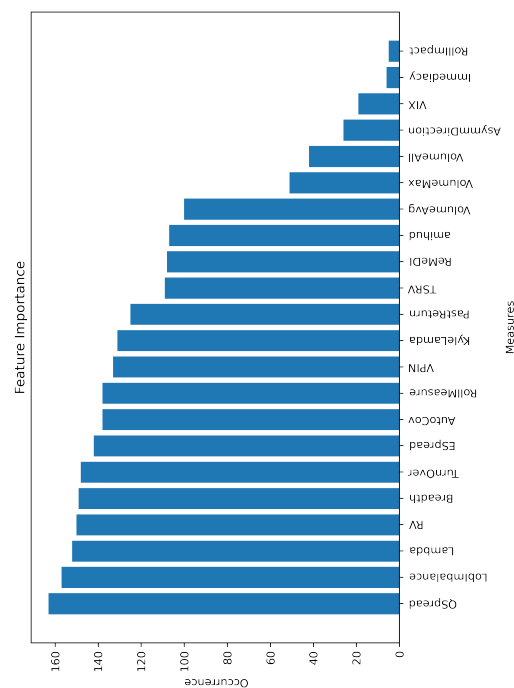


(b) 15s

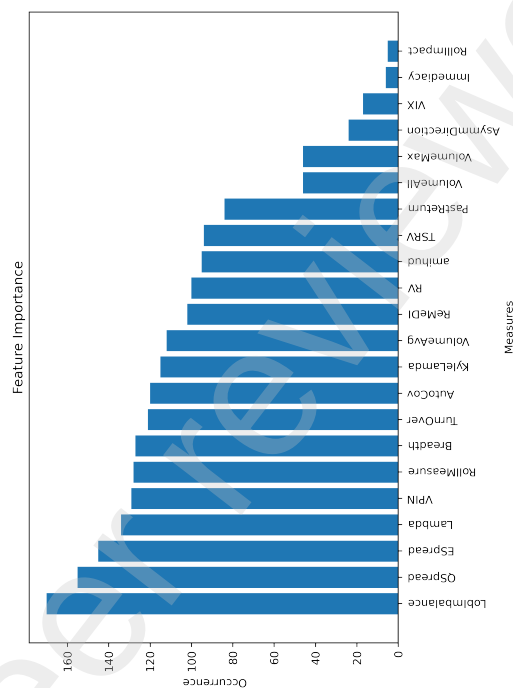


(d) 180s

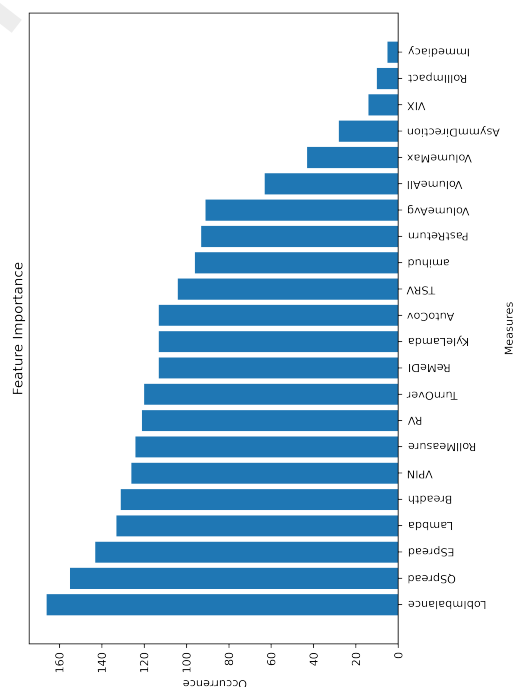
Figure 5: Features importance of mini flash crashes without reversal in Window 1



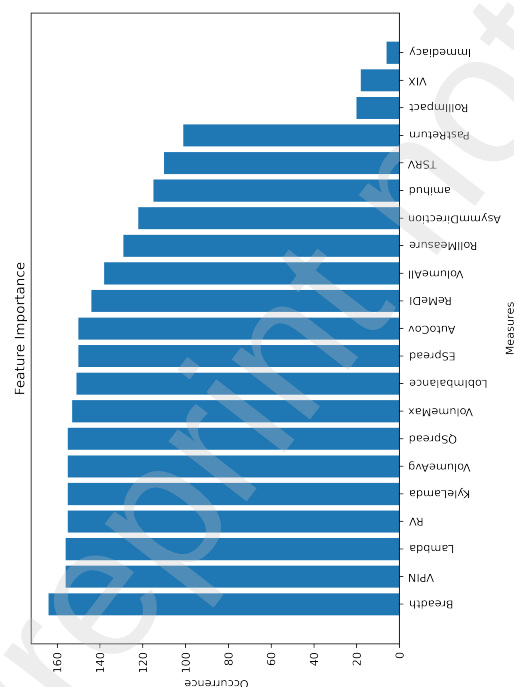
(a) 1.5s



(b) 15s

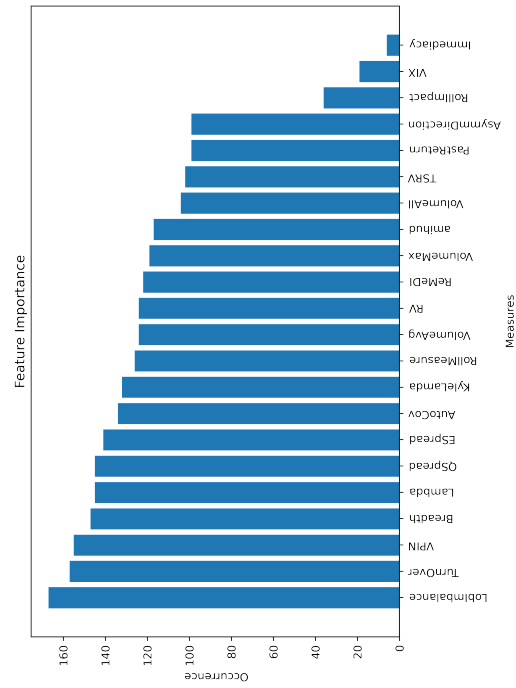


(c) 90s

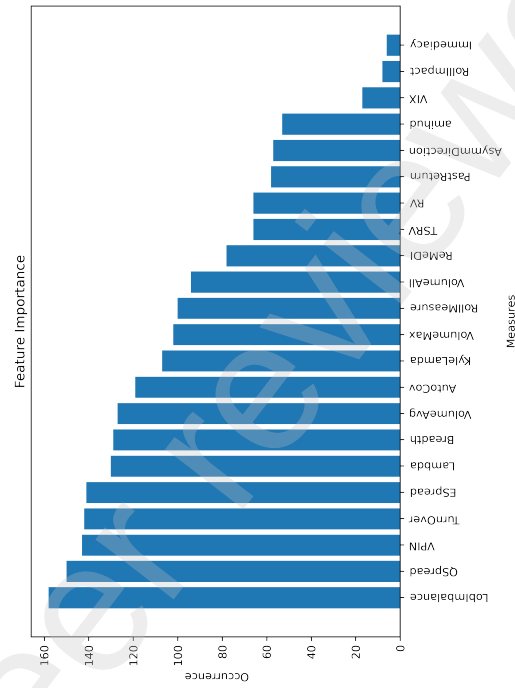


(d) 180s

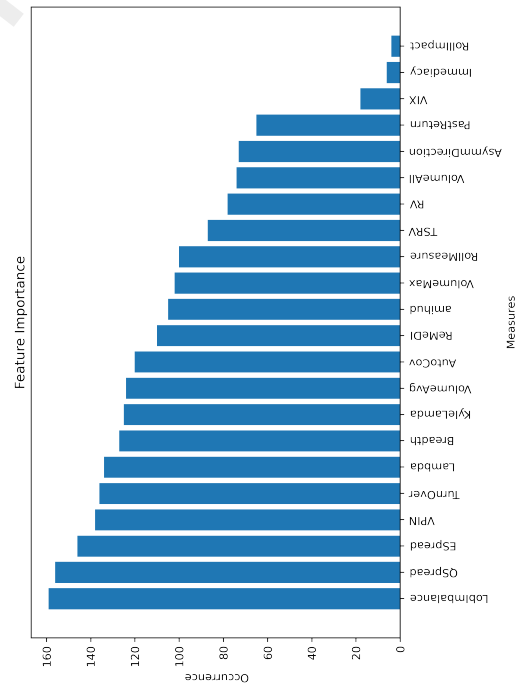
Figure 6: Features importance of mini flash crashes without reversal in Window 2



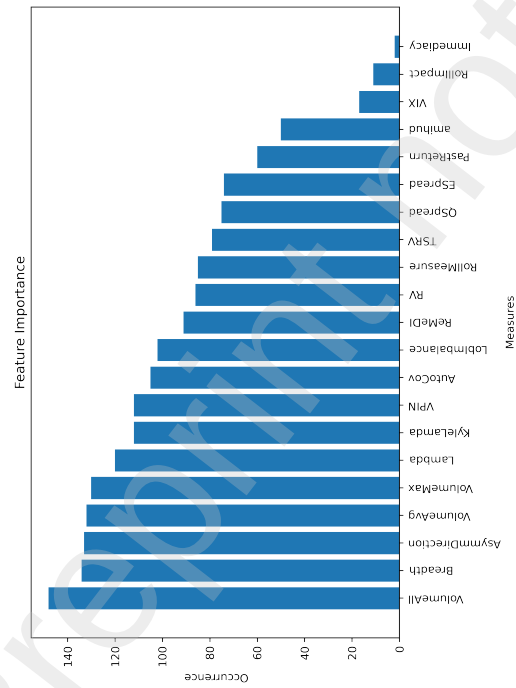
(a) 1.5s



(b) 15s

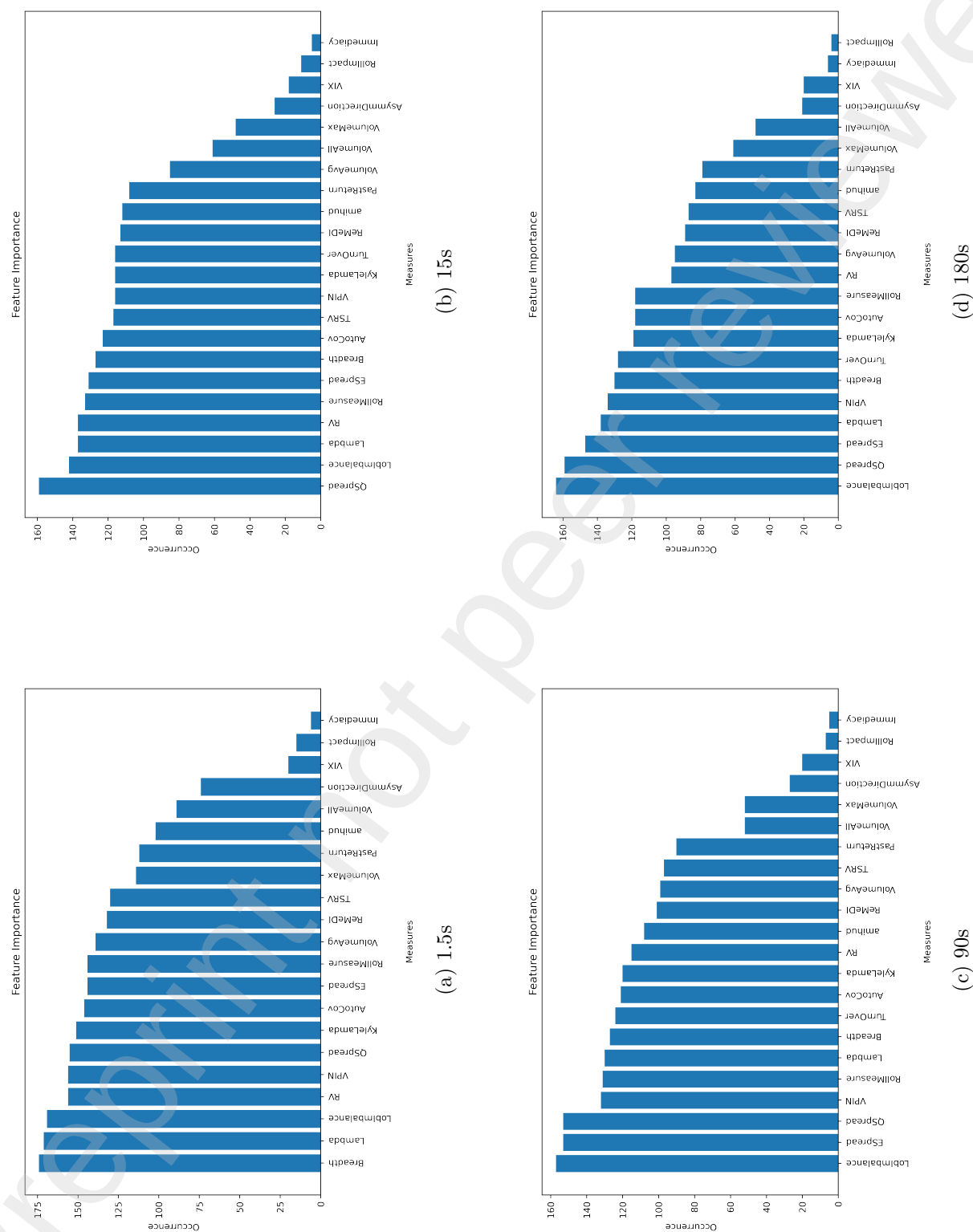


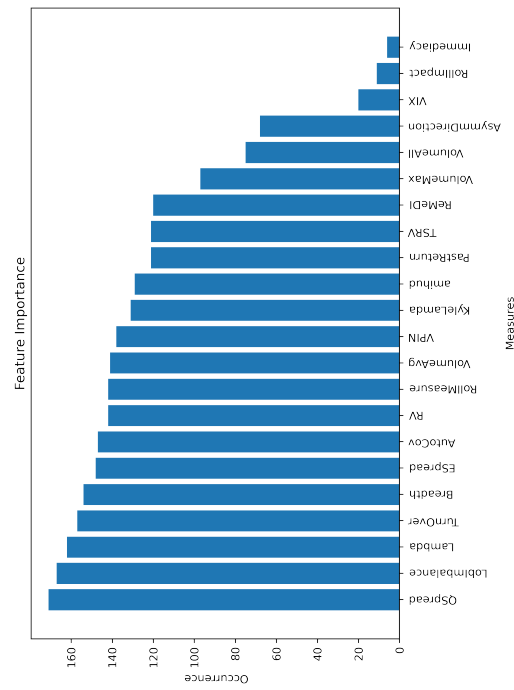
(c) 90s



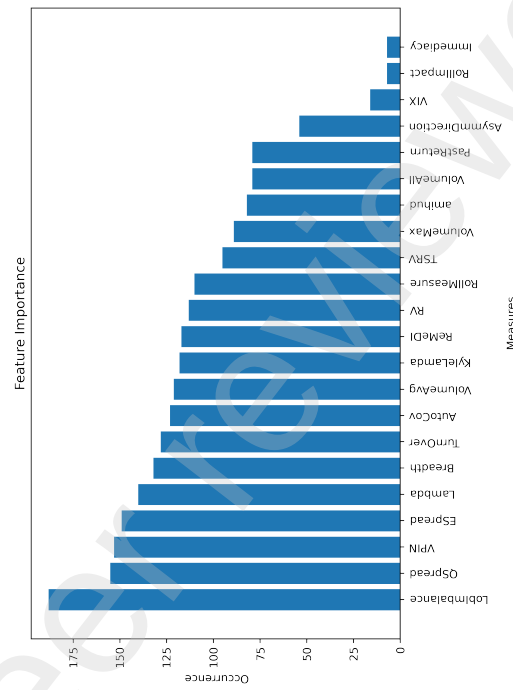
(d) 180s

Figure 7: Features importance of mini flash crashes without reversal in Window 3

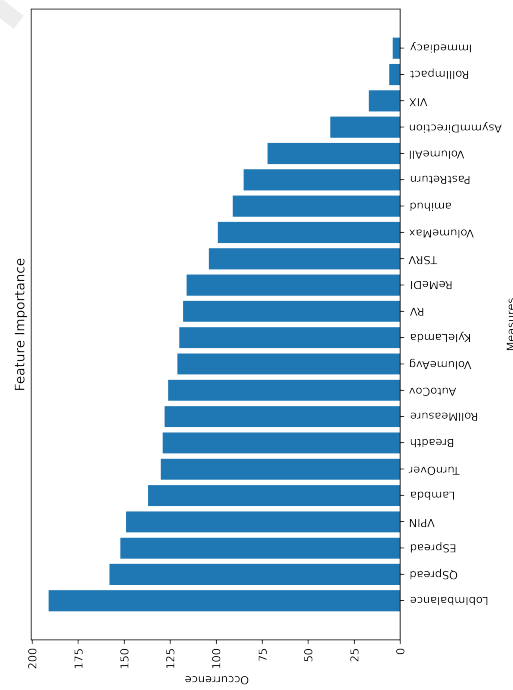




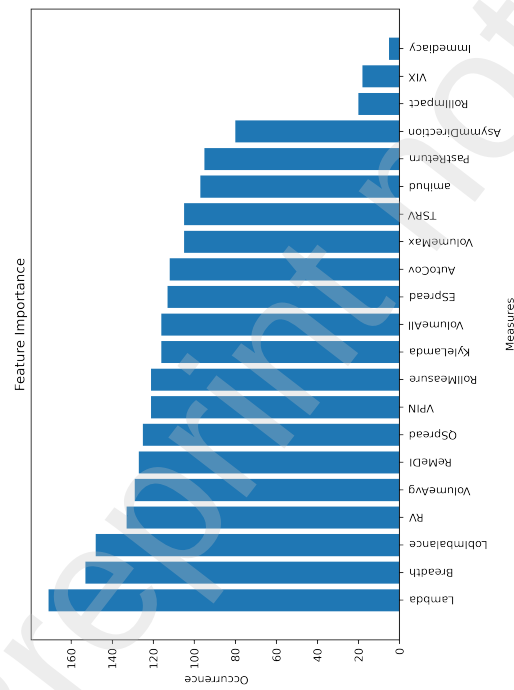
(a) 1.5s



(b) 15s

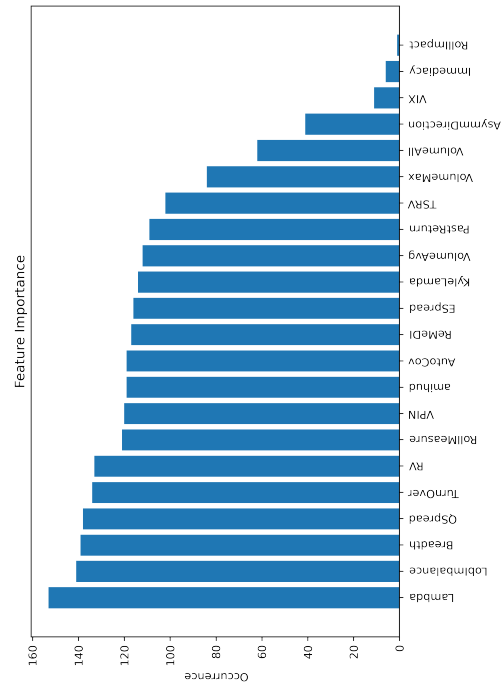


(c) 90s

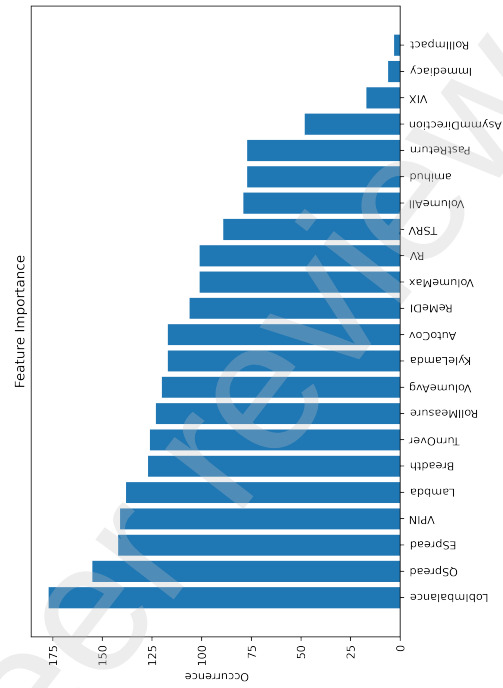


(d) 180s

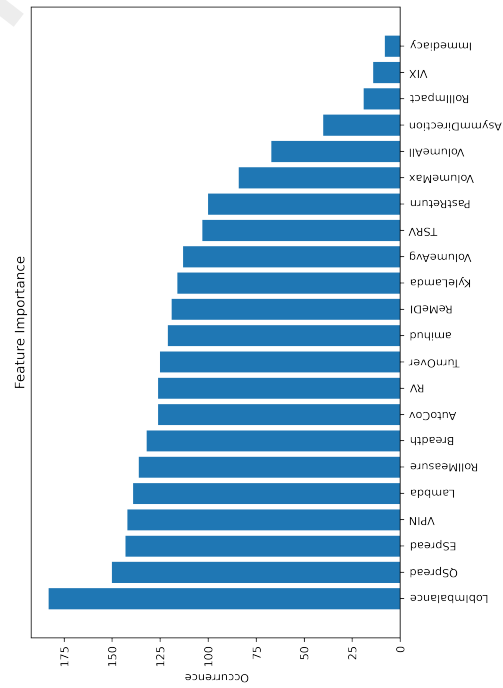
Figure 10: Features importance of mini flash crashes without reversal in Window 6



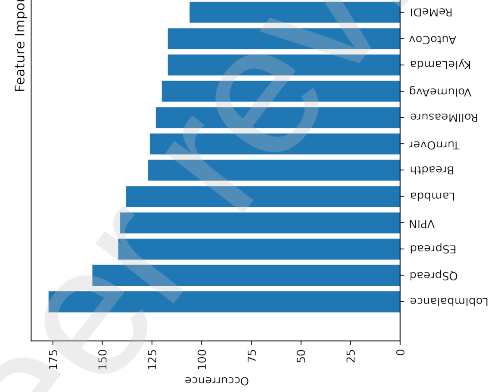
(a) 1.5s



(b) 15s

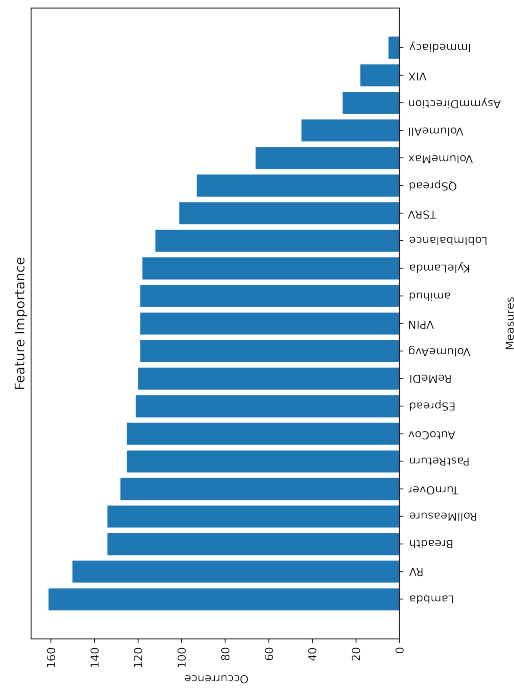


(c) 90s

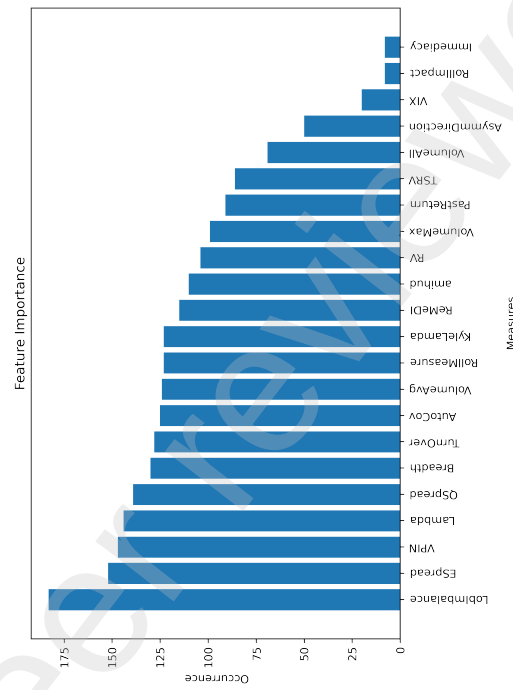


(d) 180s

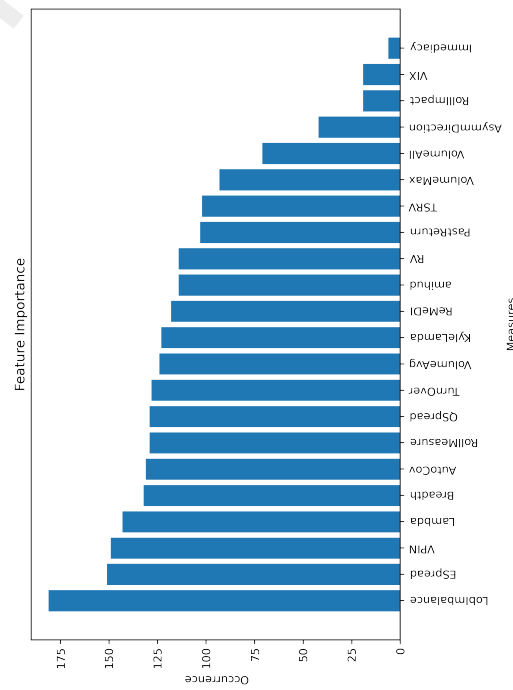
Figure 11: Features importance of mini flash crashes without reversal in Window 7



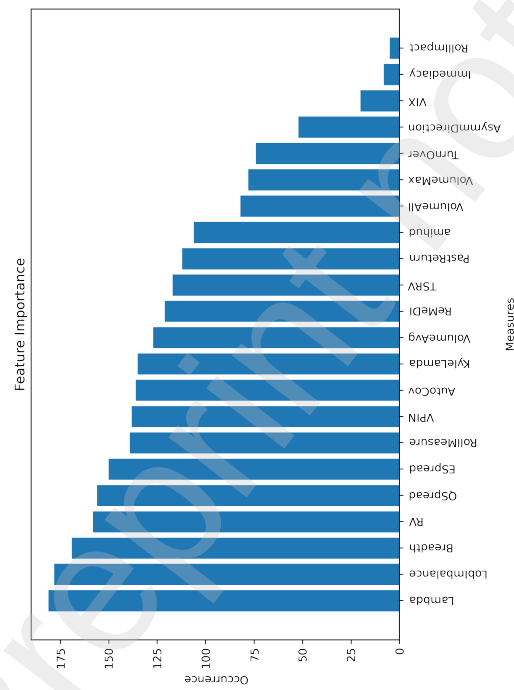
(a) 1.5s



(b) 15s

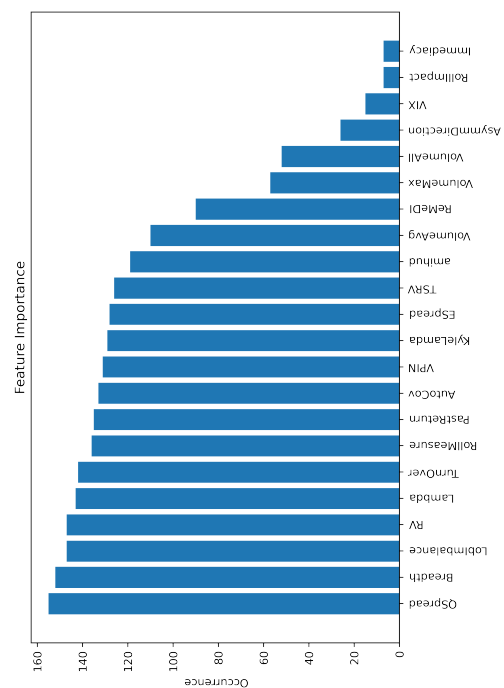


(c) 90s

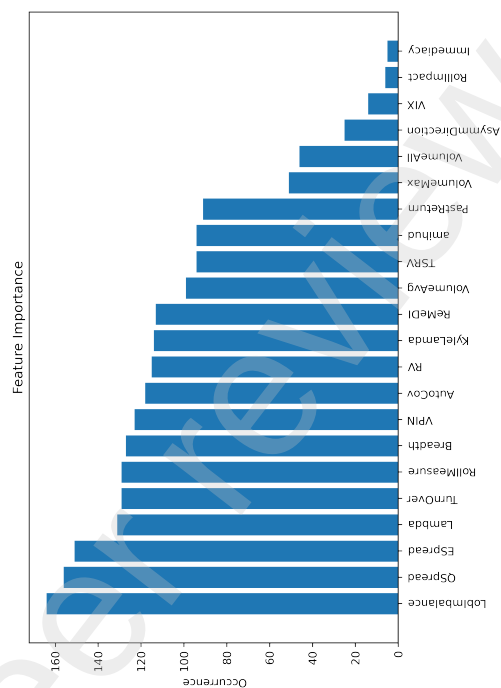


(d) 180s

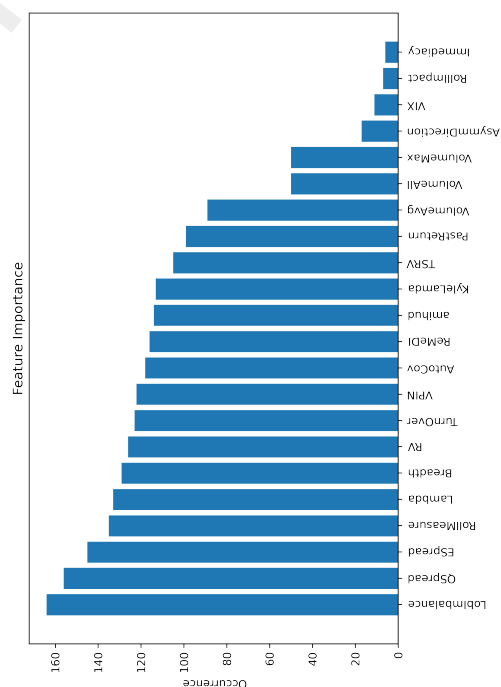
Figure 12: Features importance of mini flash crashes without reversal in Window 8



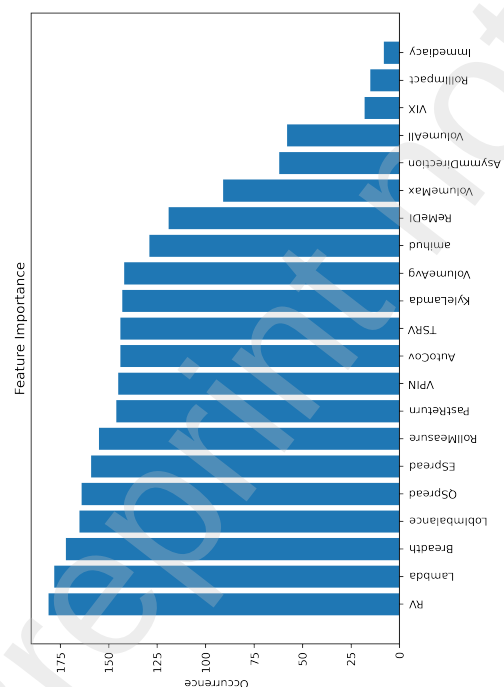
(a) 1.5s



(b) 15s

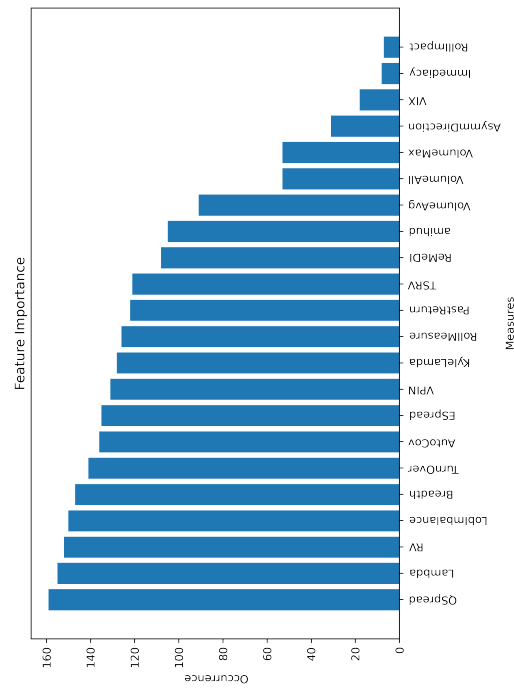


(c) 90s

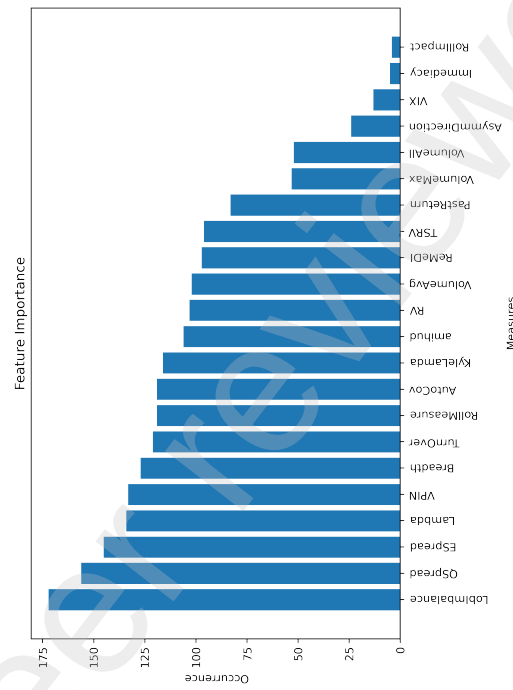


(d) 180s

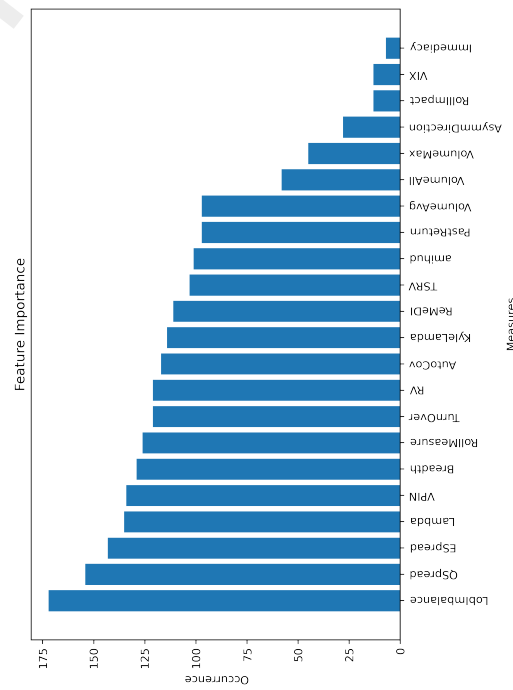
Figure 13: Features importance of mini flash crashes with reversal in Window 1



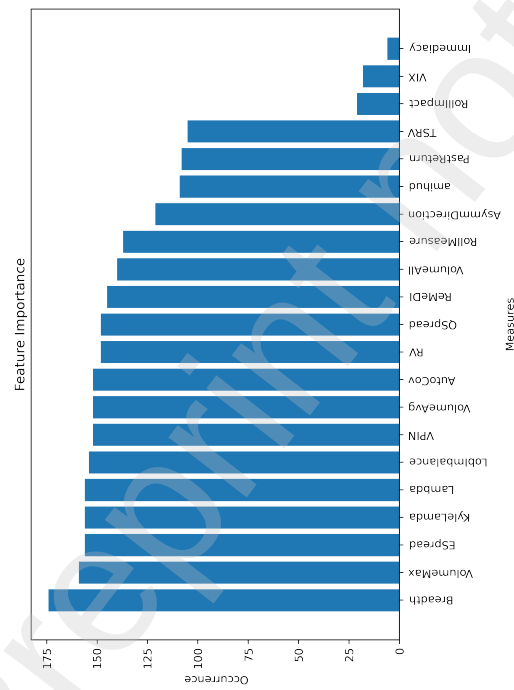
(a) 1.5s



(b) 15s

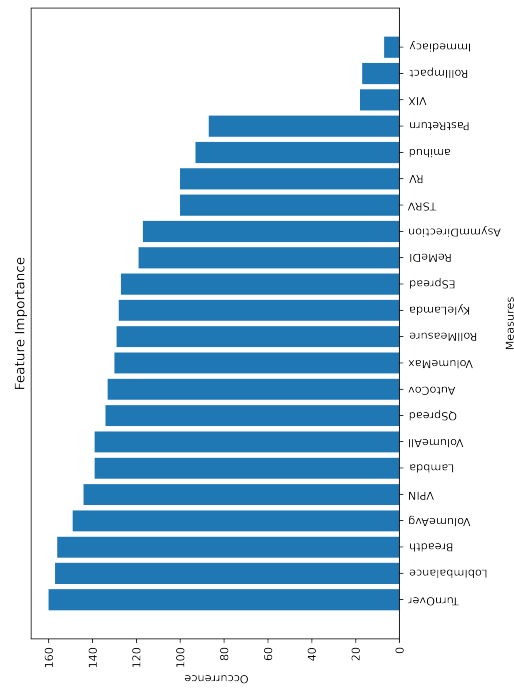


(c) 90s

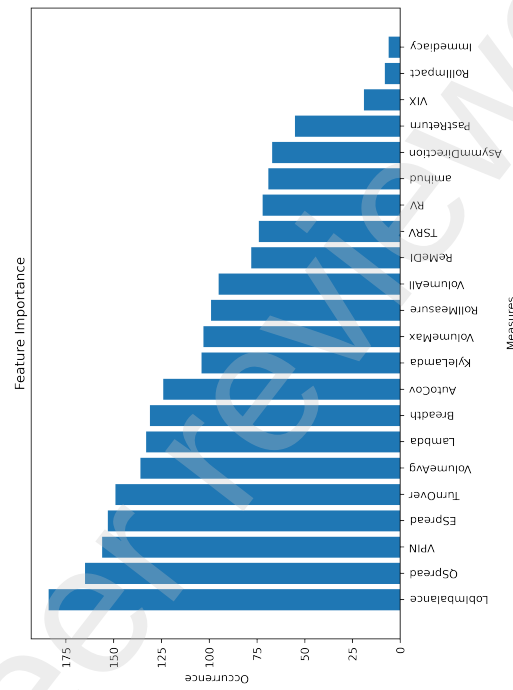


(d) 180s

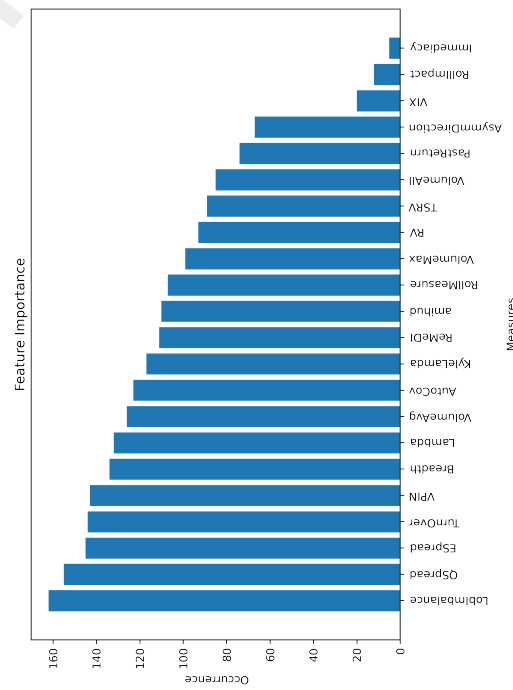
Figure 14: Features importance of mini flash crashes with reversal in Window 2



(a) 1.5s



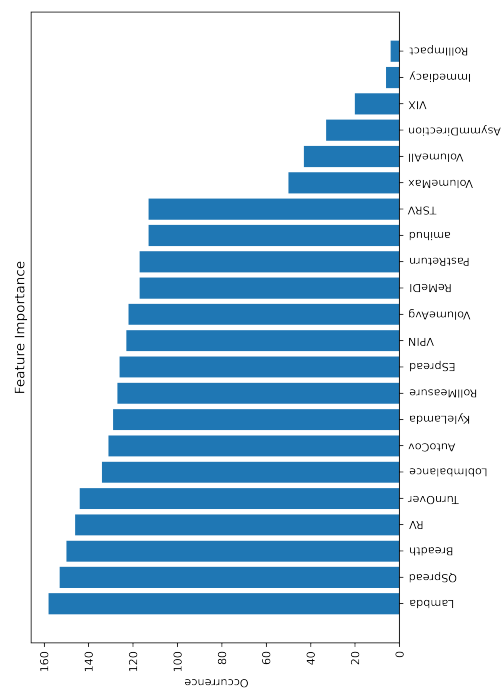
(b) 15s



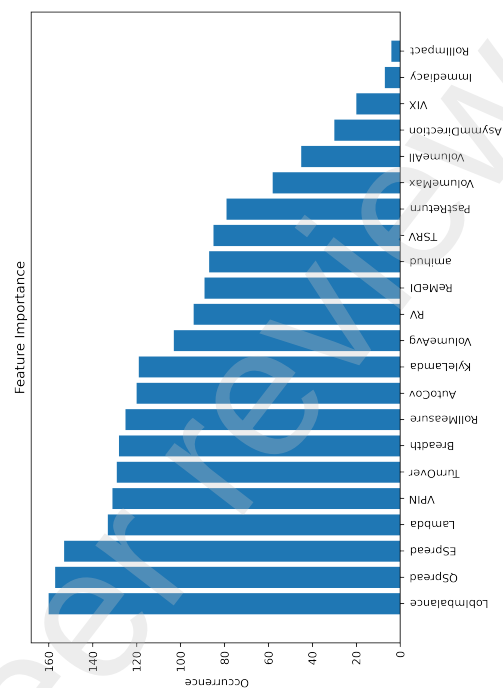
(c) 90s

(d) 180s

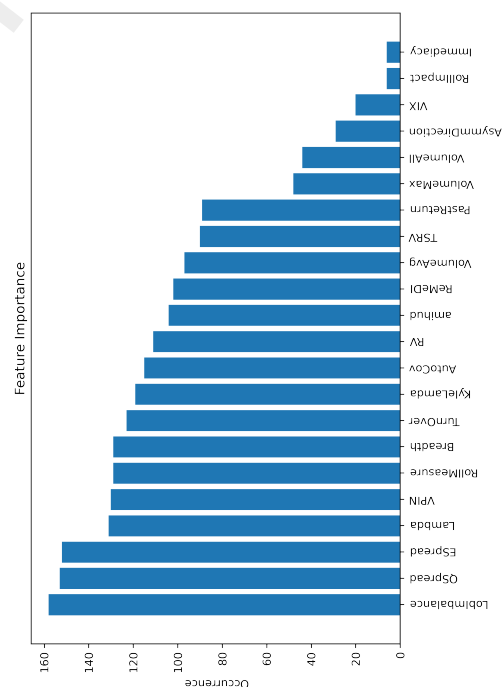
Figure 15: Features importance of mini flash crashes with reversal in Window 3



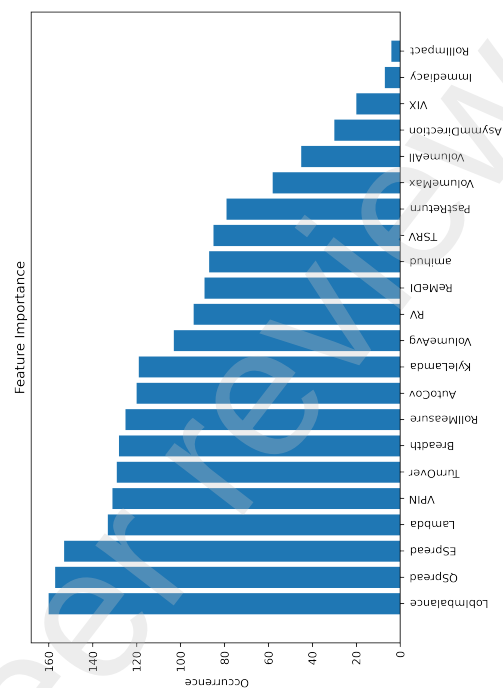
(a) 1.5s



(b) 15s

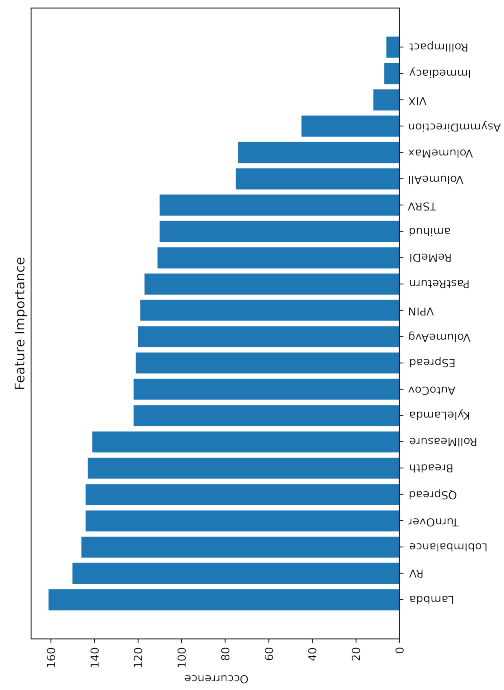


(c) 90s

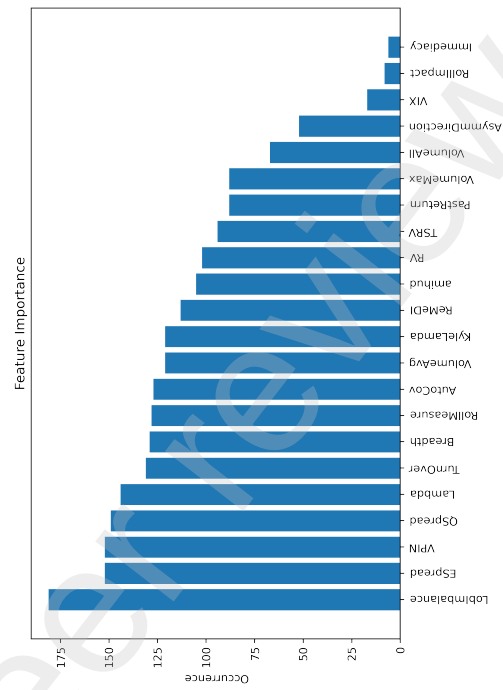


(d) 180s

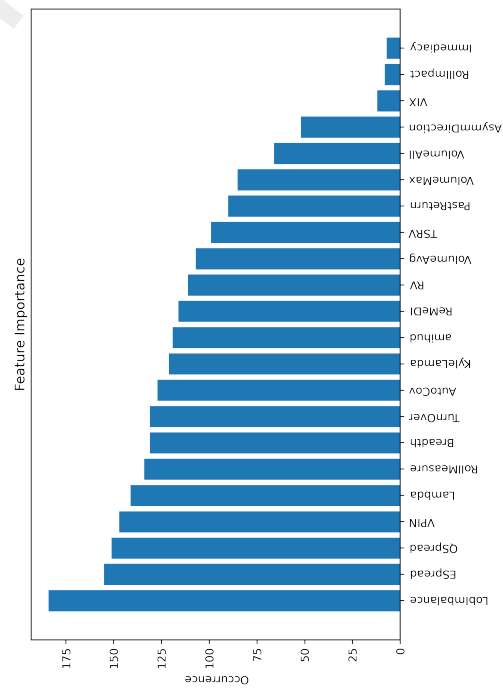
Figure 16: Features importance of mini flash crashes with reversal in Window 4



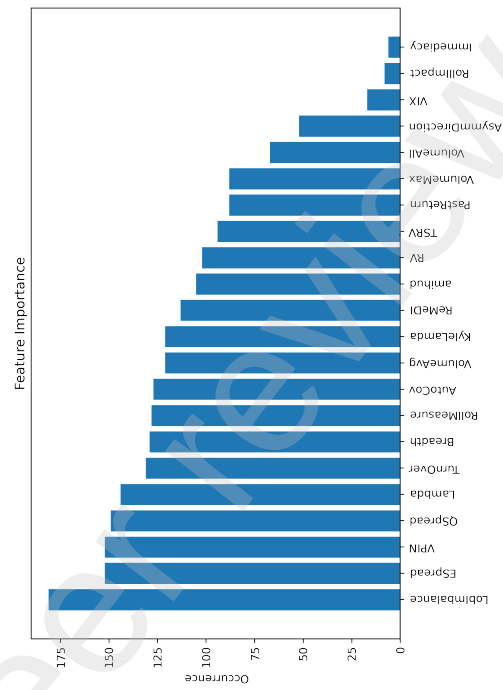
(a) 1.5s



(b) 15s

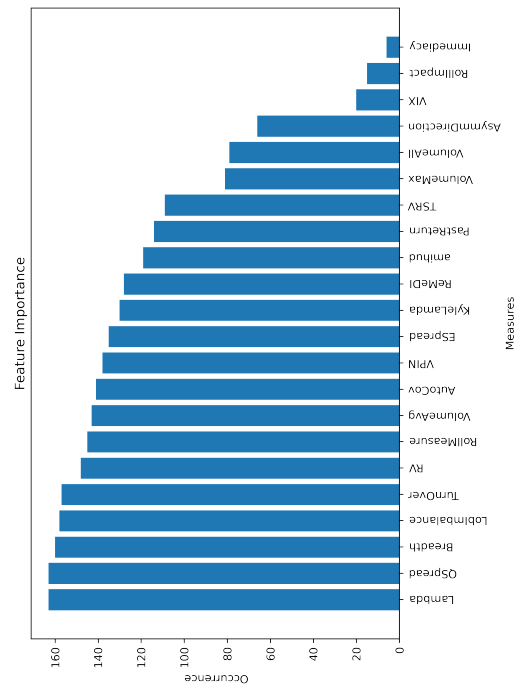


(c) 90s

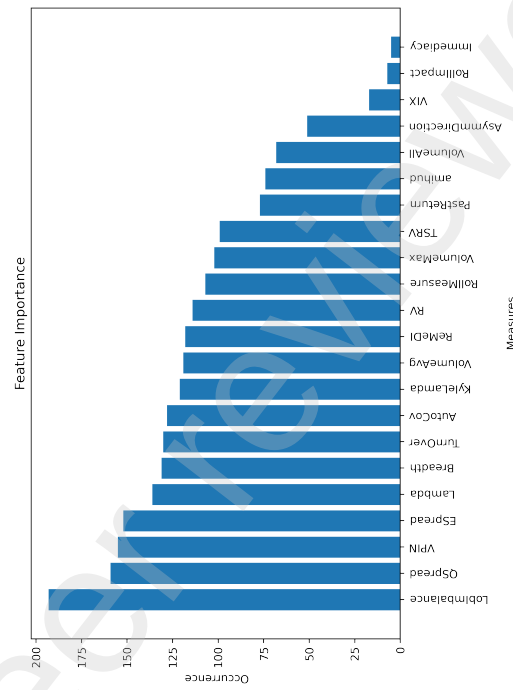


(d) 180s

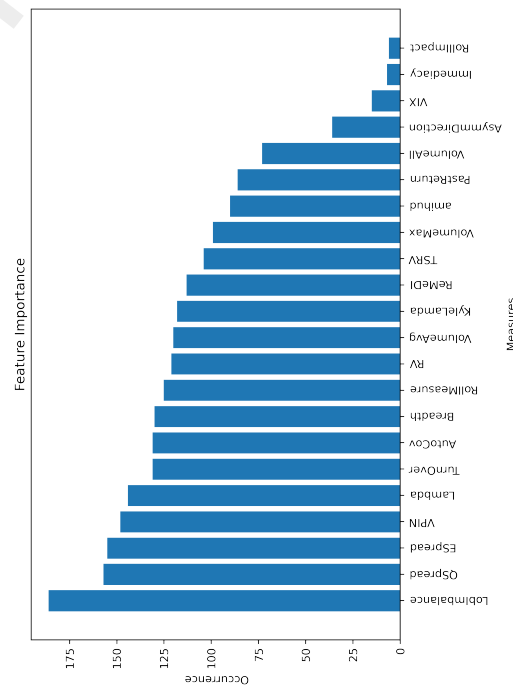
Figure 17: Features importance of mini flash crashes with reversal in Window 5



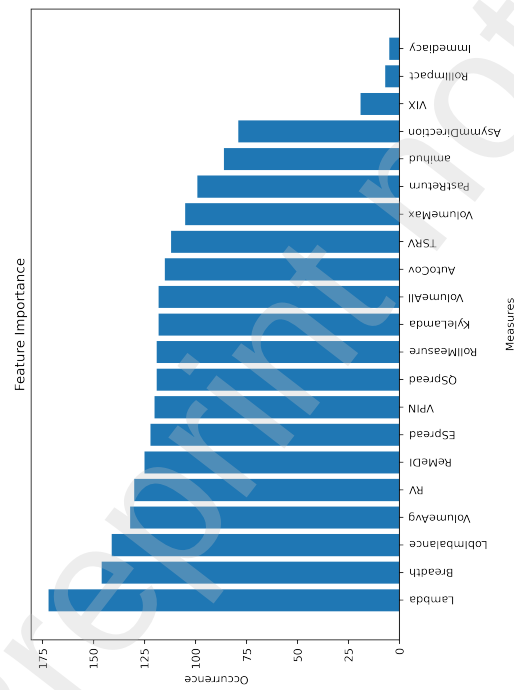
(a) 1.5s



(b) 15s

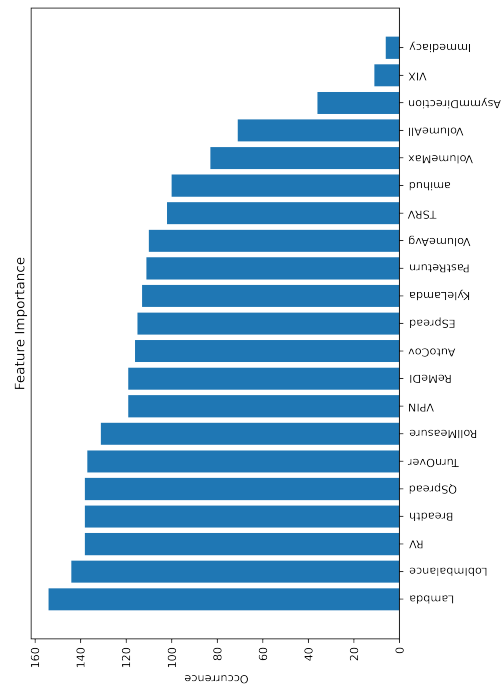


(c) 90s

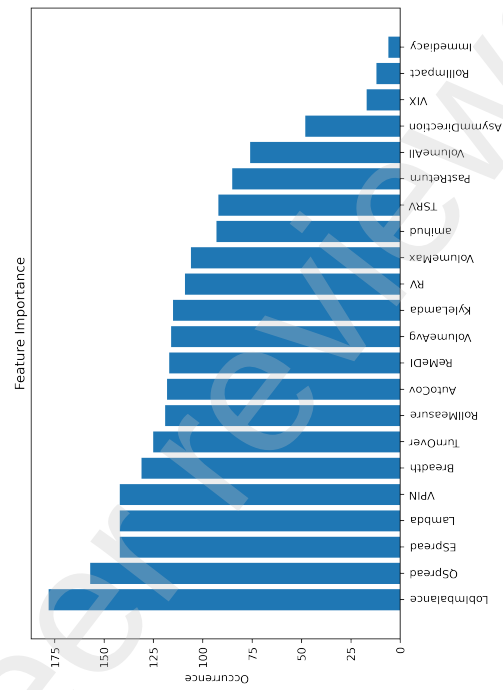


(d) 180s

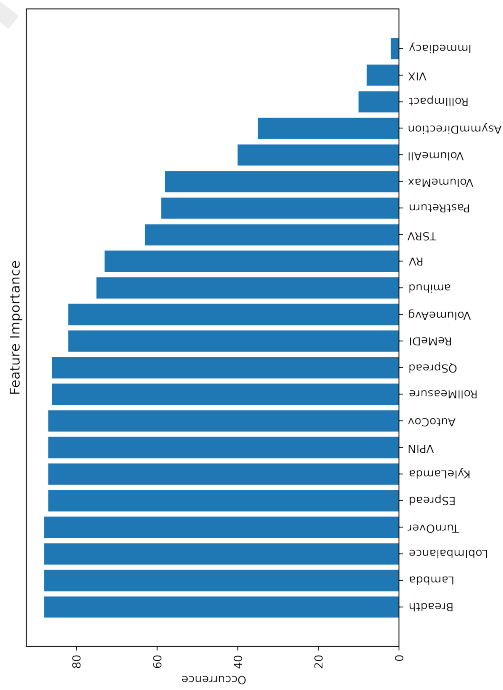
Figure 18: Features importance of mini flash crashes with reversal in Window 6



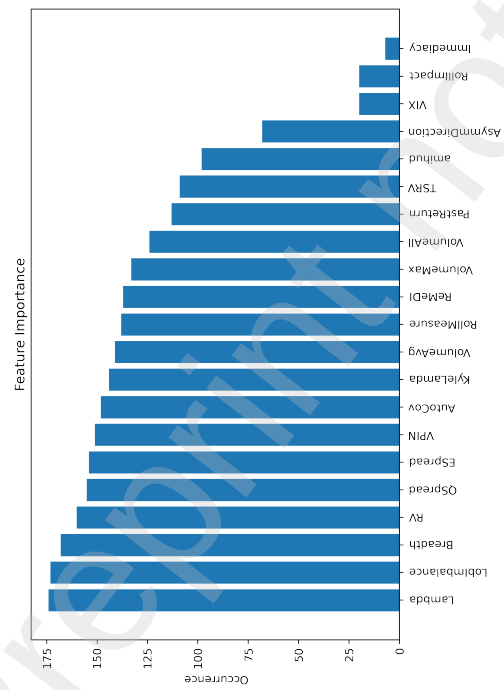
(a) 1.5s



(b) 15s

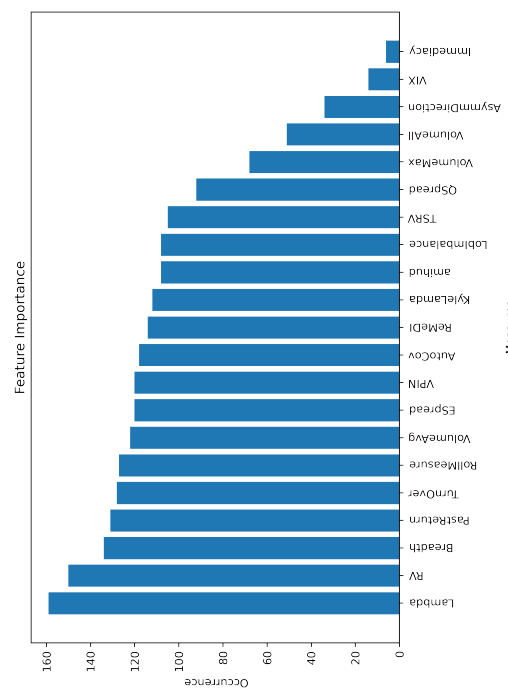


(c) 90s

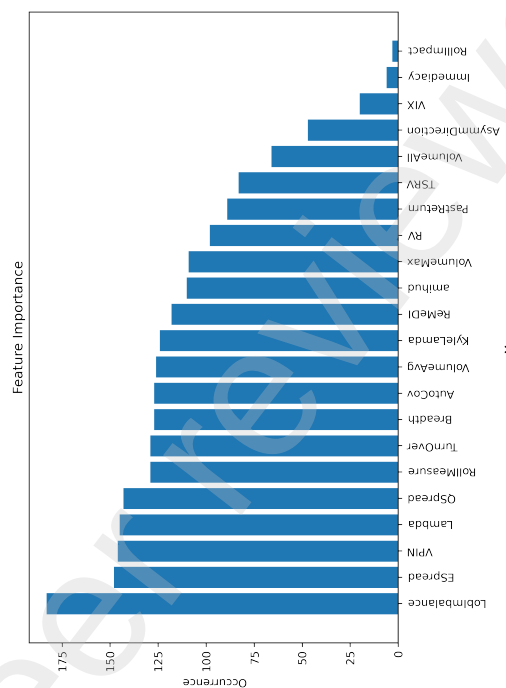


(d) 180s

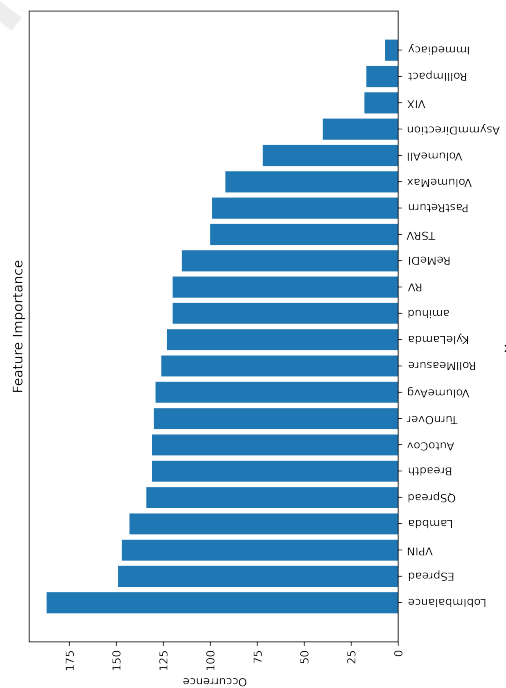
Figure 19: Features importance of mini flash crashes with reversal in Window 7



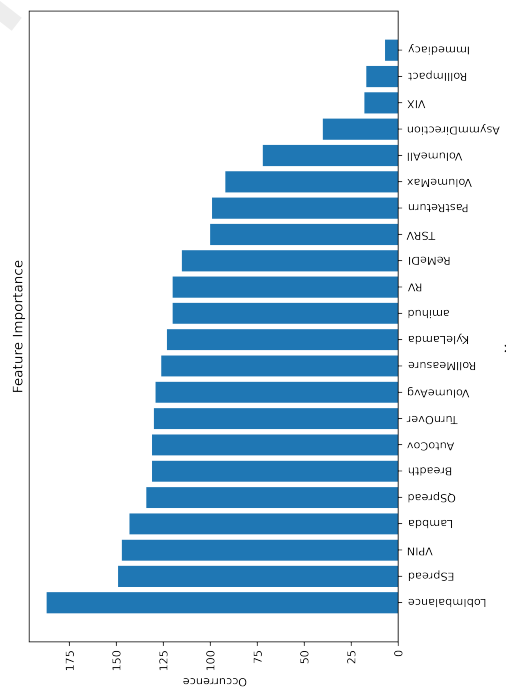
(a) 1.5s



(b) 15s



(c) 90s



(d) 180s

Figure 20: Features importance of mini flash crashes with reversal in Window 8